

Counting Cycles with AI: Computationally Efficient Equivalent Forms with Applications

Jiashun Jin, Zheng Tracy Ke, Bingcheng Sui, and Zhenggang Wang^{*†‡§}

July 21, 2026

Abstract

Cycle count statistics are fundamental tools in statistics and engineering, with applications in motif counting, channel coding, and statistical inference of network and matrix data. However, how to compute high-order cycle count statistics efficiently is still an open problem. In this paper, we aim to derive Computationally Efficient Equivalent Forms (CEEF) for cycle count statistics of any given order, where we express each cycle count statistic equivalently as a linear combination of finitely many terms. Using the CEEF, we provide a much more efficient way to compute the cycle count statistics.

The CEEF problem has no known general solution and requires delicate combinatorial arguments together with extensive calculations. While this task is hard to accomplish by humans alone, it provides an ideal setting in which Artificial Intelligence (AI) can be useful. We solve the problem by combining several theorems we derive with powerful coding skills of modern AI systems. Our results leverage graph-theoretic arguments and yield new formulas for general cases that were previously unknown. We find that, although AI cannot solve the problem independently, it becomes highly effective when guided by humans through theorems we derive as well as a clear derivation strategy, step-by-step instructions, and carefully-written prompts.

We consider several statistical applications, including spiked matrix testing, estimation of weak spike eigenvalues, and pairwise network comparison. For each problem, we demonstrate that optimal statistical performance is achieved by using high-order cycle count statistics, and our CEEF formulas make their computation feasible on large-scale data sets.

1 Introduction

Cycle count statistics are fundamental tools in engineering, information theory, statistics, and machine learning. For instance, cycle counting is a special case of motif counting and plays an important role in biomolecular engineering [3, 38], especially in the studies of protein-protein interaction networks. In channel coding, cycle counts in the Tanner graph are used as important performance metrics for the codes [23, 32]. In information theory and statistics, cycle counts and the general sub-graph counts have been used to establish information-theoretic bounds and derive optimal algorithms, such as in graph alignment [36, 50], network data inference [4, 9], and covariance matrix testing [12, 43].

Despite their importance, computing high-order cycle count statistics remains a major computational challenge. For any $n \times n$ matrix and $m \geq 3$, the order- m cycle count statistics associated with the matrix is a big sum of $O(n^m)$ terms. Seemingly, the brute-forth approach where we enumerate all $O(n^m)$ terms is infeasible when m is large. Such a challenge motivates the CEEF problem: how to derive a **C**omputationally **E**fficient **E**quivalent **F**orm (CEEF) for cycle count statistics, that is, an exact formula for the order- m cycle count statistic which allows us to compute it with a complexity much lower than $O(n^m)$. CEEF formulas of this kind only exist for small orders [11, 24, 26, 39, 41],

^{*}J. Jin is with the Department of Statistics and Data Science, Carnegie Mellon University, Pittsburgh, PA 15213, USA (e-mail: jiashun@stat.cmu.edu).

[†]Z.T. Ke is with the Department of Statistics, Harvard University, Cambridge, MA 02138, USA (e-mail: zke@fas.harvard.edu).

[‡]B. Sui is with the School of the Gifted Young, University of Science and Technology of China, Hefei, Anhui 230026, China (e-mail: suibc@mail.ustc.edu.cn).

[§]Z. Wang is with the School of Statistics and Data Science, Southeast University, Nanjing, Jiangsu 211189, China (e-mail: zggwang@seu.edu.cn).

and their derivations rely on case-by-case arguments rather than a general strategy applicable to an arbitrary order m .

In this paper, we develop a general combinatorial framework for solving the CEEF problem for arbitrary cycle orders. The framework introduces several new graph-theoretic concepts, including Fully Annotated Multigraphs (FAMs), root FAMs, Succinct Expressive Algebraic (SEA) terms, and Incompressible Full Sum (IFS) terms. Here, a FAM is a labeled multi-graph, and each SEA term or IFS term is associated with a specific FAM. In particular, fix $m \geq 3$ and consider all FAMs induced by a size- m root FAM. We first divide them by size and then by isomorphism. This way, we divide all FAMs into a number of isomorphic classes. Picking a candidate from each class, we express the order- m cycle count statistic as a linear combination of many terms, where each term corresponds to a candidate FAM, and each coefficient depends on a few simple characteristics of the candidate FAM and the isomorphic class. Moreover, to further reduce the computation complexity, we show that each term can be reduced equivalently to either an SEA term or an IFS term.

Following the above ideas, we can derive a symbolic formula for the order- m cycle count statistic, given an arbitrary order m . The formula is mathematically correct, but to determine each term and each linear coefficient explicitly, we need to enumerate all candidate FAMs. Unfortunately, such a task is merely too hard for human. Take $m = 12$ for example. The resulting CEEF formula consists of 1,900 terms, each corresponding to a candidate FAM with a coefficient to be determined. Enumerating all 1,900 FAMs are extremely labor-intensive and prone to error, making the task nearly impossible for human to accomplish.

Recent advances in Artificial Intelligence (AI) provide a promising alternative for carrying out this kind of task. In particular, for problems involving delicate and tedious combinatorial arguments, AI may have certain advantages over humans. However, a straightforward use of AI is insufficient: current AI systems are unable to solve the CEEF problem reliably through direct prompting alone. We therefore develop a human–AI collaboration workflow in which AI is not asked to solve the CEEF problem from scratch. Instead, humans first work out a list of theorems to lay out the theoretical foundation of solving the problem. Using these theorems, humans then design a rigorous algorithm with multiple steps (which can also be viewed as a multi-step reasoning and derivation strategy), and AI is then guided to understand and execute its individual steps. In summary, humans are responsible for the high-level architecture, and AI is responsible for the specific steps; humans are responsible for mathematics, and AI is responsible for coding and execution.

This division of labor is essential. Current AI systems do not appear capable of independently solving a problem as technically demanding as CEEF, making the human contribution of a correct high-level reasoning and derivation strategy indispensable. Conversely, AI is well suited to executing many labor-intensive but well-defined subproblems once the high-level strategy is determined. We believe that this workflow is useful broadly for research-level problems in AI for Mathematics (AI4Math) [17].

The remaining part of this section is organized as follows. We first formally define the CEEF problem in Section 1.1. We then give a high-level description of our combinatorial framework and human–AI collaboration workflow in Section 1.2, positioning our work in the literature, and emphasizing why the resulting mathematical and algorithmic contributions are nontrivial. Finally, we discuss the connection between our work and recent developments in AI4Math in Section 1.3.

1.1 The CEEF problem

For any symmetric matrix $A \in \mathbb{R}^{n \times n}$ and integer $m \geq 3$, the order- m cycle count (CC) statistic is defined by

$$C_m = \sum_{1 \leq i_1, \dots, i_m, (\text{dist}) \leq n} A_{i_1 i_2} A_{i_2 i_3} \dots A_{i_m i_1}, \quad \text{where ‘dist’ stands for ‘distinct’}. \quad (1)$$

Because of the restriction that $i_1, \dots, i_m$ should be distinct, the above definition does not depend on the diagonal of A . Without loss of generality, we assume

$$A_{ii} = 0, \quad 1 \leq i \leq n. \quad (2)$$

In the special case where A is the adjacency matrix of an undirected network (which we refer to as the *binary case*), $C_m/(2m)$ is the number of m -cycles in the network [26]. This is why we call C_m the term *cycle count statistics*.

Table 1: The formula of C_8 from our approach. The term marked by $\star\star$ is an IFS term, and the remaining terms are SEA terms.

+36	$\mathbf{1}'_n \cdot ((A \circ A \circ A \circ A \circ A \circ A \circ A \circ A) \cdot \mathbf{1}_n)$	-96	$\mathbf{1}'_n \cdot (((A \circ A) \cdot \mathbf{1}_n) \circ ((A \circ A \circ (A \cdot A) \circ (A \cdot A)) \cdot \mathbf{1}_n))$
-96	$\mathbf{1}'_n \cdot (((A \circ A) \cdot \mathbf{1}_n) \circ ((A \circ A \circ A \circ A \circ A \circ A) \cdot \mathbf{1}_n))$	-4	$\mathbf{1}'_n \cdot (((A \circ A) \cdot ((A \circ A) \cdot ((A \circ A) \cdot \mathbf{1}_n))) \circ ((A \circ A) \cdot \mathbf{1}_n))$
-36	$\mathbf{1}'_n \cdot (((A \circ A \circ A \circ A) \cdot \mathbf{1}_n) \circ ((A \circ A \circ A \circ A) \cdot \mathbf{1}_n))$	-24	$\mathbf{1}'_n \cdot ((A \circ A \circ (A \cdot d((A \circ A) \cdot \mathbf{1}_n) \cdot A) \circ (A \cdot A)) \cdot \mathbf{1}_n)$
-112	$\mathbf{1}'_n \cdot ((A \circ A \circ A \circ A \circ A \circ ((A \circ A) \cdot (A \circ A))) \cdot \mathbf{1}_n)$	-32	$\mathbf{1}'_n \cdot ((A \circ A \circ ((A \circ ((A \cdot A) \cdot A)) \cdot (A \circ A))) \cdot \mathbf{1}_n)$
+32	$\mathbf{1}'_n \cdot ((A \circ A \circ A \circ A \circ A \circ ((A \cdot A) \cdot A)) \cdot \mathbf{1}_n)$	-64	$\mathbf{1}'_n \cdot ((A \circ A \circ ((A \circ (A \cdot A)) \cdot (A \circ (A \cdot A)))) \cdot \mathbf{1}_n)$
+72	$\mathbf{1}'_n \cdot (((A \circ A) \cdot \mathbf{1}_n) \circ ((A \circ A) \cdot \mathbf{1}_n)) \circ ((A \circ A \circ A \circ A) \cdot \mathbf{1}_n)$	-16	$\mathbf{1}'_n \cdot ((A \circ A \circ (((A \cdot A) \circ (A \cdot A)) \cdot (A \circ A))) \cdot \mathbf{1}_n)$
+16	$\mathbf{1}'_n \cdot ((A \circ A \circ A \circ A \circ ((A \circ A) \cdot \mathbf{1}_n)) \circ ((A \circ A \circ A \circ A) \cdot \mathbf{1}_n))$	+8	$\mathbf{1}'_n \cdot ((A \circ A \circ A \circ (((A \cdot A) \cdot A) \cdot A) \cdot A)) \cdot \mathbf{1}_n)$
+80	$\mathbf{1}'_n \cdot ((A \circ A \circ A \circ A \circ (A \cdot A) \circ (A \cdot A)) \cdot \mathbf{1}_n)$	+16	$\mathbf{1}'_n \cdot (((A \circ A) \cdot \mathbf{1}_n) \circ ((A \circ A) \cdot \mathbf{1}_n)) \circ ((A \circ ((A \cdot A) \cdot A)) \cdot \mathbf{1}_n))$
+32	$\mathbf{1}'_n \cdot (((A \circ A) \cdot \mathbf{1}_n) \circ ((A \circ A \circ A \circ A) \cdot ((A \circ A) \cdot \mathbf{1}_n)))$	+8	$\mathbf{1}'_n \cdot (((A \circ A) \cdot ((A \circ A) \cdot \mathbf{1}_n)) \circ ((A \circ ((A \cdot A) \cdot A)) \cdot \mathbf{1}_n))$
+192	$\mathbf{1}'_n \cdot ((A \circ A \circ A \circ ((A \circ A \circ (A \cdot A)) \cdot A)) \cdot \mathbf{1}_n)$	+16	$\mathbf{1}'_n \cdot (((A \circ A) \cdot \mathbf{1}_n) \circ ((A \circ (A \cdot A)) \cdot \mathbf{1}_n)) \circ ((A \circ (A \cdot A)) \cdot \mathbf{1}_n))$
+32	$\mathbf{1}'_n \cdot ((A \circ A \circ A \circ ((A \cdot A) \cdot (A \circ A \circ A))) \cdot \mathbf{1}_n)$	+24	$\mathbf{1}'_n \cdot ((A \circ A \circ ((A \cdot A) \cdot A) \cdot A) \circ (A \cdot A)) \cdot \mathbf{1}_n)$
+4	$\mathbf{1}'_n \cdot ((A \circ A \circ A \circ ((A \cdot (A \circ A \circ A)) \cdot A)) \cdot \mathbf{1}_n)$	+12	$\mathbf{1}'_n \cdot ((A \circ A \circ ((A \cdot A) \cdot A) \circ (A \cdot (A \cdot A))) \cdot \mathbf{1}_n)$
+64	$\mathbf{1}'_n \cdot (((A \circ A) \cdot \mathbf{1}_n) \circ ((A \circ A \circ ((A \circ A) \cdot (A \circ A))) \cdot \mathbf{1}_n))$	+8	$\mathbf{1}'_n \cdot (((A \circ A) \cdot \mathbf{1}_n) \circ ((A \circ ((A \cdot A) \cdot A)) \cdot ((A \circ A) \cdot \mathbf{1}_n)))$
+5	$\mathbf{1}'_n \cdot ((A \circ A \circ ((A \circ A) \cdot ((A \circ A) \cdot (A \circ A)))) \cdot \mathbf{1}_n)$	+16	$\mathbf{1}'_n \cdot (((A \circ (A \cdot A)) \cdot ((A \circ A) \cdot \mathbf{1}_n)) \circ ((A \circ (A \cdot A)) \cdot \mathbf{1}_n))$
+22	$\Sigma_{i_1 i_2 i_3 i_4} (A \circ A)_{i_1 i_2} A_{i_1 i_3} A_{i_1 i_4} A_{i_2 i_3} A_{i_2 i_4} (A \circ A)_{i_3 i_4} \quad \star\star$	+4	$\mathbf{1}'_n \cdot (((A \circ A) \cdot \mathbf{1}_n) \circ ((A \circ ((A \cdot A) \cdot d((A \circ A) \cdot \mathbf{1}_n) \cdot A)) \cdot \mathbf{1}_n))$
-16	$\mathbf{1}'_n \cdot (((A \circ A \circ A \circ A) \cdot \mathbf{1}_n) \circ ((A \circ ((A \cdot A) \cdot A)) \cdot \mathbf{1}_n))$	+4	$\mathbf{1}'_n \cdot (((A \circ A) \cdot ((A \circ (A \cdot A)) \cdot \mathbf{1}_n)) \circ ((A \circ (A \cdot A)) \cdot \mathbf{1}_n))$
-64	$\mathbf{1}'_n \cdot (((A \circ (A \cdot A)) \cdot \mathbf{1}_n) \circ ((A \circ A \circ A \circ (A \cdot A)) \cdot \mathbf{1}_n))$	+24	$\mathbf{1}'_n \cdot ((A \circ ((A \cdot A) \cdot A) \circ (A \cdot A) \circ (A \cdot A)) \cdot \mathbf{1}_n)$
-8	$\mathbf{1}'_n \cdot ((A \circ A \circ A \circ (A \cdot d((A \circ (A \cdot A)) \cdot \mathbf{1}_n) \cdot A)) \cdot \mathbf{1}_n)$	+2	$\mathbf{1}'_n \cdot ((A \circ ((A \cdot A) \circ (A \cdot A) \circ (A \cdot A)) \cdot A)) \cdot \mathbf{1}_n)$
-64	$\mathbf{1}'_n \cdot ((A \circ A \circ A \circ ((A \cdot A) \cdot A)) \cdot ((A \circ A) \cdot \mathbf{1}_n))$	-8	$\mathbf{1}'_n \cdot (((A \circ A) \cdot \mathbf{1}_n) \circ ((A \circ (((A \cdot A) \cdot A) \cdot A) \cdot A)) \cdot \mathbf{1}_n))$
-16	$\mathbf{1}'_n \cdot ((A \circ A \circ A \circ ((A \cdot A) \cdot d((A \circ A) \cdot \mathbf{1}_n) \cdot A)) \cdot \mathbf{1}_n)$	-8	$\mathbf{1}'_n \cdot (((A \circ ((A \cdot A) \cdot A) \cdot A) \cdot \mathbf{1}_n) \circ ((A \circ (A \cdot A)) \cdot \mathbf{1}_n))$
+1	$\mathbf{1}'_n \cdot ((A \circ (((A \cdot A) \cdot A) \cdot A) \cdot A) \cdot A) \cdot \mathbf{1}_n)$	-4	$\mathbf{1}'_n \cdot (((A \circ ((A \cdot A) \cdot A)) \cdot \mathbf{1}_n) \circ ((A \circ (A \cdot A) \cdot A)) \cdot \mathbf{1}_n))$
-16	$\mathbf{1}'_n \cdot (((A \circ A) \cdot ((A \circ A) \cdot \mathbf{1}_n)) \circ ((A \circ A) \cdot \mathbf{1}_n)) \circ ((A \circ A) \cdot \mathbf{1}_n))$	-12	$\mathbf{1}'_n \cdot (((A \circ A) \cdot \mathbf{1}_n) \circ ((A \circ A) \cdot \mathbf{1}_n)) \circ ((A \circ A) \cdot \mathbf{1}_n)) \circ ((A \circ A) \cdot \mathbf{1}_n))$

As a brute-force approach to computing C_m , we may use m nested for-loops, but this incurs a computational complexity of $O(n^m)$. Alternatively, we may derive a formula that expresses C_m as a linear combination of many easy-to-compute terms. Such a formula can substantially reduce the computation cost. Below, we present two examples. Let $\mathbf{1}_n$ denote the n -dimensional vector of ones. For any $n \times n$ matrices B and C , let $d(B)$ denote the diagonal matrix whose diagonal entries are the same as those in B , and let $B \circ C$ represent their Hadamard (or entry-wise) product, while $B \cdot C$ denotes the matrix multiplication.

Example 1. When $m = 4$, $C_4 = \mathbf{1}'_n \cdot (A \circ A \circ A \circ A) \cdot \mathbf{1}_n - 2[\mathbf{1}'_n \cdot (((A \circ A) \cdot \mathbf{1}_n) \circ ((A \circ A) \cdot \mathbf{1}_n))] + \mathbf{1}'_n \cdot (A \circ ((A \cdot A) \cdot A) \cdot \mathbf{1}_n)$, where each term on the right hand side (RHS) can be computed with a computational complexity of $O(n^3)$.

Example 2. When $m = 8$, C_8 is a linear combination of 44 terms, as shown in Table 1. Among them, 43 ones are similar to those in Example 1 and can be computed with a complexity of $O(n^3)$, and the remaining one (marked with $\star\star$) has the form:

$$\sum_{1 \leq i_1, i_2, i_3, i_4 \leq n} (A \circ A)_{i_1 i_2} A_{i_1 i_3} A_{i_1 i_4} A_{i_2 i_3} A_{i_2 i_4} (A \circ A)_{i_3 i_4}. \quad (3)$$

This term can be computed with a complexity of $O(n^4)$.

From these examples, C_4 can be computed with a cost at most $O(n^3)$, and C_8 can be computed with a cost at most $O(n^4)$. In both cases, the complexity is much lower than $O(n^m)$ —the complexity of the brute-forth enumeration.

To formalize the CEEF problem, we introduce a few definitions.

Definition 1 (SEA term). We call a term a *Succinct Expressive Algebraic (SEA)* term if it is an elementary function of A and $\mathbf{1}_n$ constructed using only three operations: (i) multiplication (applied to two matrices or a matrix and a vector), (ii) entry-wise product (applied to two matrices or two vectors), and (iii) diagonal extraction (applied to a matrix).

In Examples 1-2, all terms in C_4 are SEA terms; and in the formula of C_8 , 43 out of the 44 terms are SEA terms.

Definition 2 (FS term). We call a term a K -layer *Full Sum (FS)* term if it has the form of

$$\sum_{1 \leq i_1, i_2, \dots, i_K \leq n} X_{j_1, k_1}^{(1)} X_{j_2, k_2}^{(2)} \dots X_{j_N, k_N}^{(N)}, \quad K \geq 1, N \geq 1,$$

where each $X^{(s)}$ is an $n \times n$ matrix that is an elementary function of A constructed using only matrix multiplication, entry-wise product, and diagonal extraction, and $j_1, k_1, \dots, j_N, k_N$ take values in $\{i_1, i_2, \dots, i_K\}$ with $j_\ell \neq k_\ell$, $1 \leq \ell \leq N$.

Definition 3 (IFS term). We say a K -layer FS term is an *Incompressible Full Sum (IFS) term* if it cannot be written equivalently as an L -layer FS term for any $L < K$.

In the example of C_8 , the term in (3) is a 4-layer FS term, where $N = 6$, $X^{(s)} = A$ for $s \in \{2, 3, 4, 5\}$, $X^{(s)} = A \circ A$ for $s \in \{1, 6\}$, and $(j_1, k_1, \dots, j_6, k_6) = (i_1, i_2, i_1, i_3, i_1, i_4, i_2, i_3, i_2, i_4, i_3, i_4)$. It can be shown that this term cannot be written as an L -layer FS term with $L < 4$, so it is an IFS term.

Any SEA term can be computed with a cost of $O(n^3)$. Any K -layer FS term, when computed directly using for-loops, has a cost of $O(n^K)$. Therefore, for $K \geq 4$, it is desirable to convert a K -layer FS term into an SEA term or an IFS term, whichever possible. If such an FS term can be converted into an SEA term, the cost is reduced from $O(n^K)$ to $O(n^3)$. Alternatively, if a K -layer FS term can be converted into a K' -layer IFS term with $K' < K$, the cost is reduced from $O(n^K)$ to $O(n^{K'})$. Moreover, by definition, an IFS term cannot be compressed into fewer layers, suggesting that its computational cost is difficult to reduce further. The discussion here motivates the CEEF problem we formulate as follows.

Definition 4 (CEEF). For each $m \geq 3$, a *Computationally Efficient Equivalent Form (CEEF)* of C_m refers to a formula that decomposes C_m into a linear combination of finitely many SEA terms and IFS terms, with explicit linear coefficients

To solve the CEEF problem, we seek an algorithm that takes an integer $m \geq 3$ as input and automatically outputs a formula of C_m satisfying Definition 4. An example of the output of our algorithm with $m = 8$ is given in Table 1.

Remark 1 (CEEF is more than a cycle-counting algorithm). Solving CEEF is fundamentally different from developing algorithms for counting fixed-length cycles, such as DFS/backtracking, Johnson’s algorithm, BFS, color coding, and dynamic programming (e.g., see [45] for a review). These algorithms take a given data matrix A as input and compute the numerical value of $C_m(A)$. However, they do not derive an algebraically equivalent expression for $C_m(A)$ that is valid for arbitrary A . Moreover, some algorithms (e.g., color coding) provide only approximate values of $C_m(A)$. Even exact algorithms are primarily designed to reduce the computational cost, without establishing that the resulting complexity is optimal. In contrast, the CEEF framework explicitly requires every retained term to be computationally irreducible, meaning that its computational complexity cannot be reduced further while preserving algebraic equivalence. In summary, the value of CEEF extends beyond efficient computation: it provides a general closed-form representation of the cycle count statistic, which is also useful for analyzing the theoretical properties of $C_m(A)$, assessing the computational efficiency of existing and future cycle-counting algorithms, and designing new approximate computation formulas that have both high accuracy and low complexity (see Remark 4).

1.2 Literature review, our new combinatorial framework, and the human-AI collaboration workflow

In our setting, A is an arbitrary $n \times n$ symmetric matrix, including the binary case as a special case. When m is relatively small, the CEEF problem can be *solved by hand*. For example, [24] solved the binary case for $m = 4, 5$, [11] solved the binary case for $m = 6$, and [39] solved the binary case for $m = 7$. The CEEF problem for general, non-binary matrices A is more difficult and has received less attention; in this setting, [26] solved $m = 3, 4$.

Unfortunately, this problem becomes much more challenging for larger m . Deriving formulas for large m requires delicate and extremely tedious calculations, where mistakes are easy to make. Indeed, [24] contained an error for $m = 7$, as later pointed out by [41]. Consequently, deriving CEEF formulas *by hand* quickly becomes infeasible as m increases. In a remarkable paper, [41] studied the case of binary A and derived formulas for C_m for $3 \leq m \leq 13$. However, their work focused exclusively on the binary case and the range $3 \leq m \leq 13$. Moreover, because some important proof details were omitted, it remains unclear how to generalize their approach to the more general setting where A is not binary and m is arbitrary. For these reasons, the CEEF problem is both technically difficult and extremely tedious for humans, and therefore remains largely unsolved.

We may tackle the CEEF problem with AI. However, as mentioned earlier, if we directly ask AI to generate the formula for a given m , the output is usually far from correct. We have also used multi-shot learning (e.g., [7, 51]), in which we provide AI with the formulas for some small m and ask it to reason

for the formulas for larger m . Unfortunately, because the formulas become rapidly more complicated as m increases, this approach also fails to produce satisfactory results.

Our proposal is a novel *human-guided AI (hugAI)* approach: we first derive a symbolic representation of C_m using several theorems. Next, to figure out a more explicit form for the formula, we design a clear derivation strategy, in the form of an “algorithm,” and provide step-by-step guidance to AI; each step is then carried out by combining the strengths of humans and AI. The key to the success of our approach lies in ensuring two properties: First, the derivation strategy must be *theoretically rigorous*, meaning that for any given m , if a human or an AI system correctly understands and executes the strategy, then the resulting output is guaranteed to be the desired formula of C_m . Second, the derivation strategy must be sufficiently *transparent and structured* for an AI system to understand and follow effectively.

Specifically, our algorithm has two steps. (1) *Merging step*: We fix a set of m distinct indices $S = \{i_1, i_2, \dots, i_m\}$ and let $\mathcal{G}$ be the graph with edges only between $i_1 \& i_2, i_2 \& i_3, \dots, i_m \& i_1$. The algorithm identifies all multi-graphs induced by $\mathcal{G}$ through node merging, and partitions these multi-graphs into equivalence classes, only retaining one representative in each class. Using the Möbius function [47], we show that C_m is a linear combination of FS terms (Definition 2), each corresponding to a multi-graph, where all linear coefficients are explicitly computable. (2) *Pruning step*: We relate each FS term to a Fully Annotated Multi-graph (FAM), where each layer in the sum corresponds to a node in FAM. We propose a recursive pruning procedure where ‘pruning a node’ is equivalent to ‘reducing one summation layer without changing the value of the sum’. We show that it converts each FS term equivalently to either an SEA term or an IFS term, thus providing a CEEF formula.

This algorithm satisfies the two properties we seek. First, it is theoretically rigorous, since we prove that its output yields a symbolic expression of C_m as the sum of SEA terms and IFS terms. Second, it is easy to follow by AI systems, because the algorithm is formulated entirely in the language of graph theory. Since graph theory is a field with highly structured notation and abundant training data available to modern AI systems, communicating this algorithm with AI becomes effective.

We use AI to implement this algorithm through carefully designed prompts. While many AI tools are available, we focus primarily on DeepSeek-R1 (DS), which has been shown to be competitive in solving mathematical problems (e.g., [18]). For completeness and cross validation, we also investigate several other AI systems using the same prompts; see Table 5.

Guided by our prompts, DS is able to successfully understand and execute the proposed algorithm (see Section 3 for details). As the final output, DS produces Python code such that, for any input $m \geq 3$, the code generates an explicit formula that satisfies Definition 4. This resolves the CEEF problem for general m and arbitrary symmetric matrices A .

Our contribution is two-fold. First, we solve a long-standing open problem in combinatorics—the CEEF problem. Compared with existing works, we use novel graph theory which has not been used before, and make a timely contribution by finding the formulas that are demanded in several research areas such as network analysis, estimating spiked eigenvalues, and matrix testing (see Section 4). Second, we show that AI can be a powerful research assistant for humans in tackling difficult open math problems. For difficult research-level math problems, whether AI can be truly useful is largely unclear. Our paper assures that AI can be useful with appropriate guidance and prompts, and we showcase how to use AI properly.

1.3 Connection to the recent research of AI4Math

Recently, AI for mathematics (AI4Math) has attracted increasing attention. Broadly speaking, existing research can be divided into two categories. The first category focuses on evaluating AI’s ability to assist mathematical research, where the interaction between humans and AI is primarily through direct prompting. The target problems may either have known solutions (e.g., [46, 48]) or remain open (e.g., [1, 8, 16]). The second category focuses on designing human–AI collaboration frameworks that go beyond direct prompting (e.g., [2, 34]). This line of work typically considers mathematical problems that are difficult to describe precisely in a single prompt and lack well-established proof strategies. Our work belongs to this second category.

For the CEEF problem, direct prompting alone is insufficient. Instead, substantial human input is required to develop the high-level reasoning and derivation strategy, while AI is employed to solve the resulting subproblems independently. The key novelty of our approach is the design of a reasoning and derivation strategy that is mathematically rigorous, not directly implied by existing results in combinatorics, and readily interpretable by AI. Together, these properties enable an effective collaboration

between humans and AI. We believe that this paradigm of human–AI collaboration has the potential to extend to many other research-level mathematical problems.

Recent advances in AI for Science (AI4Sci) have shifted the focus from improving the intrinsic capabilities of AI models to augmenting them with external tools for solving complex scientific problems [5, 21, 25, 31, 35, 37, 44, 49, 53]. Our work follows this emerging paradigm. Rather than expecting AI to solve the CEEF problem independently, we adopt a *hugAI* approach in which humans develop the reasoning and derivation strategy and AI serves as a research assistant, executing well-defined subproblems upon carefully designed prompts.

Content and notations: Section 2 presents the human-side contribution of this work, in which we present our theorems and algorithm. Section 3 presents the AI-side contribution of this work, in which we describe how to use AI to implement our algorithm; a numerical validation pipeline is also established to verify the correctness of the resulting CEEF formulas. Section 4 contains numerical experiments in three applications of the cycle count statistic, demonstrating the benefit of using higher-order C_m and thus the importance of solving CEEF. Section 5 contains a short discussion. All proofs can be found in the Appendix. Throughout this paper, $\circ$ denotes the Hadamard (or entry-wise) product. For $n > 1$, $I_n \in \mathbb{R}^{n \times n}$ denotes the identity matrix and $\mathbf{1}_n \in \mathbb{R}^n$ denotes the vector of ones. For any vector $v \in \mathbb{R}^d$, $d(v)$ or $\text{diag}(v)$ represents the $n \times n$ diagonal matrix whose i -th diagonal entry is v_i ; and for any matrix $M \in \mathbb{R}^{n \times n}$, $d(M)$ or $\text{diag}(M)$ denotes the diagonal matrix constructed from the diagonal entries of M .

2 The Algorithm and Its Theoretical Justification

This section contains the human-part contribution of the paper, including main theorems and algorithms. Discussion on the AI-part contribution is deferred to Section 3. Our goal here is to develop an “algorithm” that derives the CEEF formula of C_m for any given m . We begin in Section 2.1 by introducing a new concept in graph theory, FAM, which plays a key role in our results. The proposed algorithm consists of two steps, a merging step and a pruning step, described in Section 2.2 and Section 2.3, respectively. Both steps are formulated in the language of graph theory and are justified rigorously.

2.1 The FAM and the associated full sum

In graph theory [6, 24], a simple graph is a collection of nodes and edges such that, between any pair of distinct nodes, there is either no edge or exactly one undirected edge, and self-loops (i.e., edges connecting a node to itself) are not allowed. A multi-graph relaxes these restrictions by allowing multiple parallel edges between two nodes, as well as self-loops. In Figure 1, the left two panels illustrate the simple graph and multi-graph, respectively.

Throughout this paper, simple graphs are assumed to be undirected, whereas multigraphs may be either undirected or directed. For a directed multigraph, each edge from node a to node b is represented by a triplet (a, b, s) , where $1 \leq s \leq w(a, b)$ and $w(a, b)$ denotes the number of edges from a to b . For an undirected multigraph, edges between nodes a and b are likewise represented by triplets (a, b, s) with $1 \leq s \leq w(a, b)$. In this case, the ordering of a and b does not matter, and $w(a, b) = w(b, a)$ always holds. We write $E(\mathcal{G})$ for the set of all triplets (a, b, s) such that $w(a, b) \geq 1$.

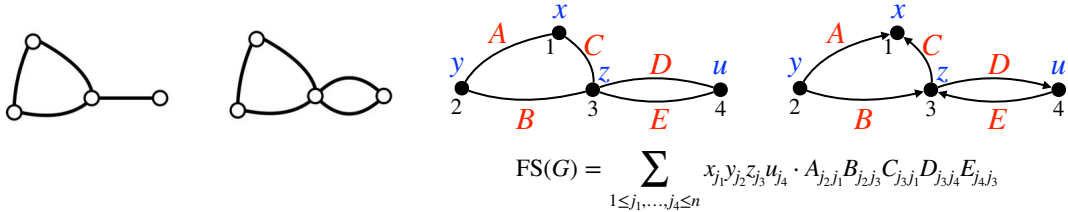


Figure 1: A simple graph (left), a multi-graph (middle left), an undirected FAM (middle right), and a directed FAM (right). In the two FAMs, nodes are labeled by $x, y, z, u \in \mathbb{R}^n$, and edges are labeled by $A, B, C, D, E \in \mathbb{R}^{n \times n}$. The corresponding Full Sums (FSs) are displayed below these graphs. For an undirected FAM, the edge-label matrices are symmetric, so the value of the FS remains the same if we replace $A_{j_2 j_1}$ by $A_{j_1 j_2}$ (in comparison, for a directed FAM, the value of the FS may change if we replace $A_{j_2 j_1}$ by $A_{j_1 j_2}$).

We propose the *Fully Annotated Multi-graph (FAM)*, as well as the Full Sum and Distinct Sum associated with the FAM. These concepts play a central role in our results.

Definition 5 (Fully Annotated Multi-graph (FAM)). A FAM $\mathcal{G}$ is a multi-graph whose nodes are labeled by vectors $v^{(a)} \in \mathbb{R}^n$ and whose edges (a, b, s) are labeled by matrices $M^{(a,b,s)} \in \mathbb{R}^{n \times n}$. For an undirected FAM, all edge-label matrices are required to be symmetric.

Definition 6 (FS and DS associated with a FAM). Given an FAM $\mathcal{G}$, the Full-Sum (FS) associated with $\mathcal{G}$ is defined as

$$\text{FS}(\mathcal{G}) := \sum_{1 \leq j_1, \dots, j_k \leq n} \left(\prod_{1 \leq a \leq k} v_{j_a}^{(a)} \prod_{(a,b,s) \in E(\mathcal{G})} M_{j_a j_b}^{(a,b,s)} \right),$$

where v_j denotes the j th entry of a vector $v \in \mathbb{R}^n$ and $M_{jj'}$ denotes the (j, j') th entry of a matrix $M \in \mathbb{R}^{n \times n}$. The Distinct-Sum (DS) associated with $\mathcal{G}$, $\text{DS}(\mathcal{G})$, is defined similarly, except that $j_1, \dots, j_k$ are required to be distinct in the sum.

In Figure 1, the right two panels illustrate an undirected FAM and a directed FAM, respectively. Both contain four nodes and six edges. Their associated FS expressions are nearly identical; the key difference is that, for an undirected FAM, terms such as $A_{j_2 j_1}$ and $B_{j_2 j_3}$ may be replaced by $A_{j_1 j_2}$ and $B_{j_3 j_2}$, respectively, without changing the value of the FS, because the edge-label matrices are symmetric. In contrast, for directed FAMs, such substitutions may change the value of the FS.

Note that $\text{FS}(\mathcal{G})$ can be viewed as a special case of the FS expression in Definition 2, in which certain matrices $X^{(s)}$ are restricted to be diagonal and the associated indices satisfy $j_s = k_s$.

The above definitions connect to our problem through a particular FAM constructed as follows. Fix $m \geq 3$ and a set of m distinct indices $S = \{i_1, i_2, \dots, i_m\}$, $1 \leq i_1, i_2, \dots, i_m \leq n$. Let $\mathcal{G} = \mathcal{G}(S)$ denote the simple graph where each index in S is a node and we have a total of m (undirected) edges between $i_1 \& i_2$, $i_2 \& i_3$, $\dots$, $i_{m-1} \& i_m$, and $i_m \& i_1$, respectively, but nowhere else. We call $\mathcal{G}$ the root graph associated with S .

Definition 7 (The root-FAM). Fix $m \geq 3$ and $S = \{i_1, i_2, \dots, i_m\}$, where $i_1, i_2, \dots, i_m$ are distinct indices in $\{1, 2, \dots, n\}$. Let $\mathcal{G}$ be the root graph as described above. We define the root FAM associated with S to be the FAM obtained by labeling each node of $\mathcal{G}$ with $\mathbf{1}_n$ and each edge with the matrix A (the original data matrix). By construction, every root FAM is undirected.

We observe that the FS and DS associated the root-FAM are

$$\text{FS}(\mathcal{G}) = \sum_{j_1, \dots, j_m} A_{j_1 j_2} A_{j_2 j_3} \dots A_{j_m j_1} = \text{trace}(A^m), \quad \text{DS}(\mathcal{G}) = \sum_{j_1, \dots, j_m (\text{dist})} A_{j_1 j_2} A_{j_2 j_3} \dots A_{j_m j_1} = C_m. \quad (4)$$

This illustrates how these concepts are intrinsically connected to our problem. Note that although $\mathcal{G}$ depends on S , the quantities $\text{FS}(\mathcal{G})$ and $\text{DS}(\mathcal{G})$ do not depend on S .

2.2 The merging process

In the merging process, we aim to express C_m as a linear combination of finitely many FS terms. In detail, let $m \geq 3$ and $S = \{i_1, i_2, \dots, i_m\}$, where $i_1, i_2, \dots, i_m$ are distinct. Recall that $\mathcal{G} = \mathcal{G}(S)$ denotes the root-FAM associated with S .

Definition 8 (Partition). Fix $k \leq m$. We call $\sigma = \{S_1, S_2, \dots, S_k\}$ a k -partition of S if $S_1, S_2, \dots, S_k$ are pairwise disjoint non-empty subsets of S satisfying $\cup_{j=1}^k S_j = S$. We call k the size of the partition (denoted by $|\sigma|$) and each S_j a block. The set of all partitions for S are denoted by $\Pi(S)$.

Given each $\sigma \in \Pi(S)$, we can create a new FAM from the root FAM, by node merging according to the partition σ .

Definition 9 (The induced FAM by a partition). Fix $1 \leq k \leq m$ and a k -partition $\sigma = \{S_1, S_2, \dots, S_k\}$. The induced FAM associated with σ is the undirected FAM with node set $\{j_1, j_2, \dots, j_k\}$. Its edges are induced by those of the root FAM as follows: an edge (a, b) in the root FAM induces an edge (j_ℓ, j_t) whenever $a \in S_\ell$ and $b \in S_t$. Consequently, the induced FAM may contain parallel edges and self-loops. Each node is still labeled by $\mathbf{1}_n$, and each edge is still labeled by the data matrix A . We call $k = |\sigma|$ the size of the induced FAM. For $1 \leq \ell \leq k$, the quantity $|S_\ell|$ is called the multiplicity of node j_ℓ .

By this definition, for each $1 \leq k \leq m$ and each k -partition, we obtain a size- k induced FAM (when $k = m$, it is the root FAM itself). We call such a process the *merging process*.

Example 3. Suppose $m = 4$ and $S = \{i_1, i_2, i_3, i_4\}$. We consider a partition $\sigma = \{\{i_1\}, \{i_2, i_4\}, \{i_3\}\}$. The induced FAM has three nodes $\{j_1, j_2, j_3\}$, where j_2 is obtained from merging i_2 and i_4 . After merging, there are two parallel edges between j_1 and j_2 , and two parallel edges between j_2 and j_3 ; all node labels are $\mathbf{1}_n$ and all edge labels are A . The associated FS term is

$$\text{FS}(\mathcal{G}_\sigma) = \sum_{j_1, j_2, j_3} (A_{j_1 j_2})^2 (A_{j_2 j_3})^2 = \sum_{j_1, j_2, j_3} (A \circ A)_{j_1 j_2} (A \circ A)_{j_2 j_3} = \mathbf{1}'_n (A \circ A)^2 \mathbf{1}_n.$$

There are a total of 9 partitions for $m = 4$. However, many of the induced FAMs have self-loops. Since A has a zero diagonal, the associated FS term of such a FAM is always zero. Therefore, we restrict our attention to those induced FAMs that have no self-loop. Figure 2 illustrates two such induced FAMs.

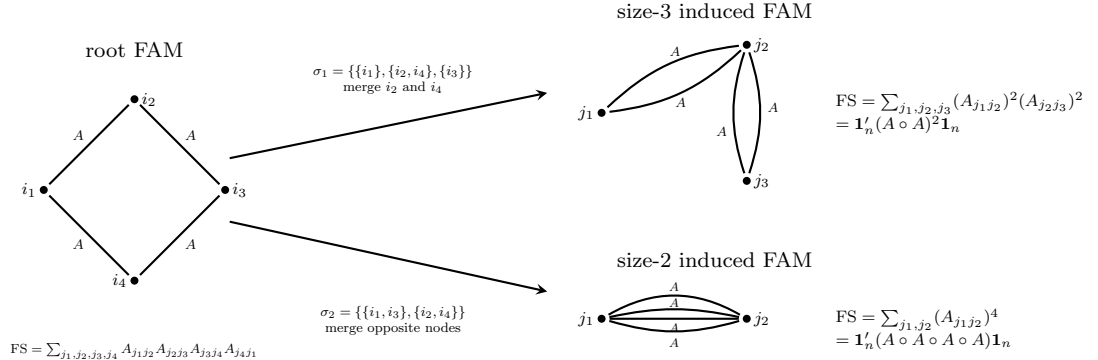


Figure 2: A partial illustration of the merging process for $m = 4$. The process starts from the root FAM (a 4-cycle) and considers all partitions. Each partition yields an induced FAM. Only two induced FAMs that have no self-loops are shown here.

Definition 10. We say two induced FAMs are isomorphic if one can be obtained from the other by a permutation of the nodes.

The merging process identifies all induced FAMs for the root FAM. Some of these induced FAMs are isomorphic. When two FAMs are isomorphic, it follows by Definition 6 that their corresponding FS terms must be equal. We thus divide all these induced FAMs into different classes, first by size and then by isomorphism. In detail, we propose Algorithm 1.

We present our main theorem about the merging step.

Theorem 1. Fix $m \geq 3$. Let $\mathcal{G}_{m,k,t}$, $d_{m,k,t}$ and $h_{m,k,t}$ be as in Algorithm 1 for $2 \leq k \leq m$ and $1 \leq t \leq b_{m,k}$. Let $\sigma_{m,k,t}$ be the partition that induces the FAM $\mathcal{G}_{m,k,t}$. Define $\mu(\hat{0}, \sigma_{m,k,t}) = (-1)^{m-k} \cdot h_{m,k,t}$ and $a_{m,k,t} = d_{m,k,t} \cdot \mu(\hat{0}, \sigma_{m,k,t})$. Then,

$$C_m = \sum_{k=2}^m \sum_{t=1}^{b_{m,k}} a_{m,k,t} \cdot \text{FS}(\mathcal{G}_{m,k,t}).$$

This theorem provides a rigorous computational formula of C_m : It expresses C_m as a linear combination of finitely many FS terms, where all linear coefficients are explicitly computable. By Definition 6, $\text{FS}(\mathcal{G}_{m,k,t})$ involves k layers of sum and can be computed with $O(n^k)$ cost. Since k ranges from 2 to m , this formula alone does not reduce the computational complexity compared to the brute-forth approach. To complete the story, we will develop a pruning process in Section 2.3, which converts each $\text{FS}(\mathcal{G}_{m,k,t})$ equivalently to either an SEA term or an IFS term.

Remark 2 (Connection to the Möbius function). In Theorem 1, $\mu(\cdot, \sigma_{m,k,t})$ coincides with the Möbius function on a partition lattice, and $\hat{0}$ refers to the finest partition. Initially, we were not aware that the linear coefficients in the decomposition of C_m are related to the Möbius function. However, when we guided the AI (specifically, DeepSeek-R1) to compute these coefficients, AI hinted us that the Möbius inversion technique might be relevant. Indeed, our proof of Theorem 1 involves deriving an explicit form of the Möbius function. A more detailed discussion is contained in the Appendix.

Algorithm 1 The merging process.

Input: $S = \{i_1, i_2, \dots, i_m\}$.

- a. List all partitions in $\Pi(S)$. For each partition, plot the induced FAM and record the size.
- b. Exclude all induced FAMs with self-loops (because any induced size-1 FAM must have self-loops and are excluded, the size of the retaining induced FAMs ranges from 2 to m).
- c. Divide all remaining FAMs into $(m - 1)$ groups, where the k -th group contains all FAMs with size k , for $2 \leq k \leq m$ (when $k = m$, group k has only one member, which is the root-FAM $\mathcal{G}$).
- d. Fix $2 \leq k \leq m$. Partition the k -th group into equivalence classes such that two FAMs belong to the same class if and only if they are isomorphic. List all classes in some fixed order (e.g. lexicographical order), and let

$$\begin{aligned} b_{m,k} &= \text{the total number of classes in the } k\text{-th group,} \\ d_{m,k,t} &= \text{cardinality of class } t \text{ (i.e., the number of FAMs contained in that class),} \quad 1 \leq t \leq b_{m,k}. \end{aligned} \tag{5}$$

Now, all induced FAMs are partitioned into classes according first to their size and then to graph isomorphism. Each class is indexed by a pair (k, t) , where $2 \leq k \leq m$ and $1 \leq t \leq b_{m,k}$.

- e. For each class (k, t) , select a candidate FAM denoted by $\mathcal{G}_{m,k,t}$. Denote by $j_1, \dots, j_k$ the nodes of $\mathcal{G}_{m,k,t}$ and by g_i the degree of node j_i (in this case, g_i coincides with the number of original nodes grouped into S_i). Define $h_{m,k,t}$ by

$$h_{m,k,t} = \prod_{i=1}^k (g_i - 1)! \quad \text{if } k < m, \quad \text{and} \quad h_{m,k,t} = 1 \quad \text{if } k = m. \tag{6}$$

Output: $b_{m,k}$ for $2 \leq k \leq m$, and $(\mathcal{G}_{m,k,t}, d_{m,k,t}, h_{m,k,t})$ for $2 \leq k \leq m$ and $1 \leq t \leq b_{m,k}$.

Remark 3 (Self-loops). Algorithm 1 excludes induced FAMs that contain self-loops. Indeed, when a FAM has self-loops, each summand in the associated Full Sum (see Definition 6) involves diagonal entries of A , which are assumed to be zero in our setting (otherwise, we plug in $A - \text{diag}(A)$). As a result, the Full Sum is zero and omitted in the computation circuit.

We note that C_m is the Distinct-Sum associated with the root FAM $\mathcal{G}$; see (4). In this sense, Theorem 1 expresses a DS-term as a linear combination of FS-terms. One may wonder whether the opposite is true. The following corollary shows that $\text{tr}(A^m)$, the Full-Sum associated with the root FAM, can be expressed as a linear combination of DS-terms on the induced FAMs.

Corollary 2. *For any $m \geq 3$,*

$$C_m = \text{tr}(A^m) - \sum_{k=2}^{m-1} \sum_{t=1}^{b_{m,k}} d_{m,k,t} \cdot \text{DS}(\mathcal{G}_{m,k,t}).$$

This provides an alternative decomposition of C_m that may also of interest in theory and in practice.

Example 4. When $m = 4$, $b_{m,k} = 1$ for $k = 2, 3, 4$; i.e., each group has one equivalence class. Table 2 shows the induced FAMs that represent the three classes. Suppose $\mathcal{G}_{m,k,t}$ has nodes $\{j_1, \dots, j_k\}$. By Definition 6, there is a function $f_{m,k,t}(A, j_1, \dots, j_k)$ such that $\text{FS}(\mathcal{G}_{m,k,t}) = \sum_{j_1, \dots, j_k} f_{m,k,t}(A, j_1, \dots, j_k)$; see Table 2. For example, $f_{m,k,t}(A, j_1, \dots, j_k) = (A_{j_1 j_2})^2 (A_{j_2 j_3})^2$ for the second FAM. It follows that $\text{FS}(\mathcal{G}_{m,k,t}) = \sum_{j_1, j_2, j_3} (A_{j_1 j_2})^2 (A_{j_2 j_3})^2 = \mathbf{1}'_n (A \circ A)^2 \mathbf{1}_n$, so this term is equivalent to an SEA term. A similar observation is made for the other two FAMs. This explains the CEEF formula in Example 1.

Our results in this merging step are new, and the proofs are non-trivial. First, by introducing two new notions (FAM and the associated Full Sum for the FAM) in graph theory, we are able to decompose C_m as a linear combination of the Full Sum of many induced FAMs. This allows us to develop sophisticated graph theory and use the results to analyze C_m . In particular, the two notions play an important role in the pruning step below. Second, to figure out the linear coefficients in Theorem 1, we need to combine careful analysis with existing results on the Möbius function (see Remark 2).

Table 2: The key quantities for $m = 4$, with Full-Annotated Multi-graphs (FAM) included for illustration. Combining the following table with Theorem 1, we obtain $C_4 = \text{tr}(A^4) - 2[\mathbf{1}'_n (A \circ A)^2 \mathbf{1}_n] + \mathbf{1}'_n (A \circ A \circ A \circ A) \mathbf{1}_n$.

(k, t)	Partition	Multigraph	FAM	$(d_{mkt}, h_{mkt}, a_{mkt})$	$f_{mkt}(A, j_1, \dots, j_k)$	SEA form
$(4, 1)$	$\{\{i_1\}, \{i_2\}, \{i_3\}, \{i_4\}\}$ $j_k = i_k, 1 \leq k \leq 4$			$(1, 1, 1)$	$A_{j_1 j_2} A_{j_2 j_3} A_{j_3 j_4} A_{j_4 j_1}$	$\text{tr}(A^4)$
$(3, 1)$	$\{\{i_1\}, \{i_2, i_4\}, \{i_3\}\}$ $j_1 = i_1, j_2 = i_2, j_3 = i_3$			$(2, 1, -2)$	$(A_{j_1 j_2})^2 (A_{j_2 j_3})^2$	$\mathbf{1}'_n (A \circ A)^2 \mathbf{1}_n$
$(2, 1)$	$\{\{i_1, i_3\}, \{i_2, i_4\}\}$ $j_1 = i_1, j_2 = i_2$			$(1, 1, 1)$	$(A_{j_1 j_2})^4$	$\mathbf{1}'_n (A \circ A \circ A \circ A) \mathbf{1}_n$

2.3 The pruning process

In the merging process, C_m has been expressed as a linear combination of finitely many FS-terms, where all linear coefficients are explicitly computable. Our goal now is to convert each $\text{FS}(\mathcal{G}_{m,k,t})$ equivalently to either an SEA term or an IFS term. To the best of our knowledge, this is the first work to formally introduce the notions of SEA terms and IFS terms. In particular, the concept of SEA terms based on three fundamental matrix-vector operations has never appeared in previous studies (e.g., [11, 24, 26, 39, 41]). Consequently, no existing methodology is available for this task; and the formulas previously derived for relatively small values of m rely largely on case-by-case analysis rather than on a principled procedure. In contrast, we develop a general and principled procedure applicable to arbitrary values of m .

Fix an FAM $\mathcal{G} = \mathcal{G}_{m,k,t}$. By Definition 6, there is a one-to-one correspondence between the number of nodes in this FAM and the number of summation layers in the associated Full Sum (the latter is directly connected to computational complexity). We propose a recursive pruning algorithm, in which “pruning a node” means reducing the number of summation layers by one, while preserving the value of the sum. To achieve this goal, we develop a scheme to prune the *Type-I* and *Type-II* pendants defined as follows.

Definition 11 (Pendants). In an undirected multi-graph, a node is called a *Type-I pendant* node if it has exactly 1 immediate neighbor (node b is an immediate neighbor of a if there exists at least one edge between them), and a node is called a *Type-II pendant* node if it has exactly 2 immediate neighbors. In either case, each immediate neighbor of a *pendant* node is referred to as a *hinge* node. For a FAM, a node is called a Type-I (respectively, Type-II) pendant if it is a Type-I (respectively, Type-II) pendant in the underlying undirected multi-graph obtained by removing all edge directions and all node/edge labels.

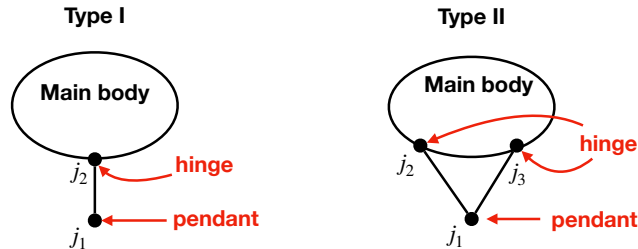


Figure 3: Type I pendant (left) and Type II pendant (right). No matter whether the FAM is directed or undirected, pendants are defined based on the underlying undirected multi-graph.

A FAM $\mathcal{G}$ obtained from the previous merging step is initially undirected. However, the pruning process requires $\mathcal{G}$ to be represented as a directed FAM. To this end, we arbitrarily assign an orientation

to each edge in $\mathcal{G}$. Because the edge-label matrices of $\mathcal{G}$ are symmetric at this stage, the assigned orientations do not affect the corresponding full sum $\text{FS}(\mathcal{G})$. As pruning proceeds, pendant nodes are successively removed and the FAM is updated accordingly. During this process, some edge-label matrices may become asymmetric, making it necessary to keep track of the edge orientations.

Definition 12 (Orientation alignment). Given a directed FAM $\mathcal{G}$, an orientation alignment from node a to node b , denoted by $\text{OA}_{a \rightarrow b}(\mathcal{G})$, re-orient every edge between a and b so that it points from a to b . Specifically, edges already directed from a to b are left unchanged, while edges directed from b to a are reversed and their corresponding edge-label matrices are replaced by their transposes.

Combining Definition 12 with Definition 6, an orientation alignment does not alter the FS-term associated with a FAM.

We now describe the pendant pruning procedure. For both Type-I and Type-II pendants, the procedure consists of two steps: first performing an orientation alignment, and then pruning the pendant node and updating the node labels and edge labels of the resulting FAM. The essential requirement is that each of these operations preserves the corresponding FS-term.

Consider a Type-I pendant node j_1 , with j_2 as the hinge node. Suppose there are s parallel edges between them. We conduct orientation alignment from the hinge node to the pendant node, i.e., $\text{OA}_{j_2 \rightarrow j_1}(\mathcal{G})$. Let $M^{(1)}, \dots, M^{(s)} \in \mathbb{R}^{n \times n}$ be the edge-labels after orientation alignment. Let $v \in \mathbb{R}^n$ and $u \in \mathbb{R}^n$ be the respective node labels for j_1 and j_2 . When we prune the pendant node, we remove j_1 and all edges between j_1 and j_2 , and update the node label of the hinge node j_2 by

$$u^{\text{new}} = u \circ [(M^{(1)} \circ \dots \circ M^{(s)}) \cdot v], \quad (7)$$

where $\circ$ is the entry-wise product between two vectors or two matrices. Aside from the above modifications, all other nodes, edges, and their labels remain unchanged. An illustration of this updating rule is provided in the left panel of Figure 4.

Consider a Type-II pendant node j_1 , with j_2 and j_3 as the hinge nodes. We perform $\text{OA}_{j_1 \rightarrow j_2}(\mathcal{G})$ and $\text{OA}_{j_1 \rightarrow j_3}(\mathcal{G})$, which means that all edges between the pendant and two hinges are re-oriented to point from the pendant to the hinges. Suppose there are s parallel edges between j_1 and j_2 , and their labels are $Q^{(1)}, \dots, Q^{(s)} \in \mathbb{R}^{n \times n}$ after orientation alignment, and there are t parallel edges between j_1 and j_3 , with labels $R^{(1)}, \dots, R^{(t)} \in \mathbb{R}^{n \times n}$ after orientation alignment. Let $y, x, z \in \mathbb{R}^n$ be the node labels for j_1, j_2, j_3 , respectively. When we prune the pendant node, we remove j_1 , all edges between j_1 and j_2 , and all edges between j_1 and j_3 ; and we add a new directed edge from j_2 to j_3 with the following label

$$E = (Q^{(1)} \circ \dots \circ Q^{(s)})^\top \cdot \text{diag}(y) \cdot (R^{(1)} \circ \dots \circ R^{(t)}). \quad (8)$$

Here, $\circ$ denotes entry-wise product, $\cdot$ denotes matrix multiplication, and $\text{diag}(y)$ is a diagonal matrix whose diagonal entries are from y . All other nodes, edges, and their labels remain unchanged. An illustration is provided in the right panel of Figure 4.

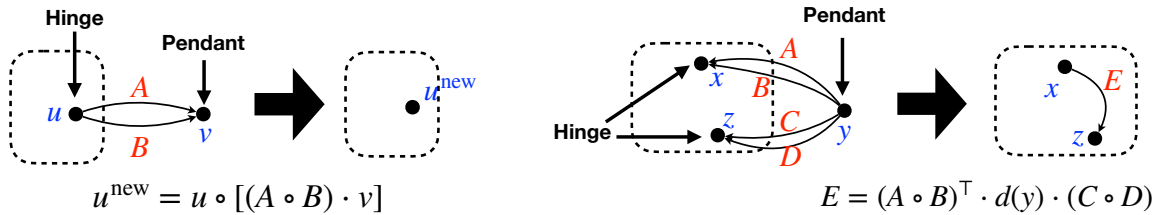


Figure 4: The pruning process and corresponding updating rule. Left: Type I. Right: Type II.

Lemma 3 (Invariance of the Full-Sum under pruning). Consider a FAM $\mathcal{G}$ with at least 1 pendant node which is either Type-I or Type-II. Suppose we pick any pendant node and prune it according to the two rules in (7)-(8), and let $\mathcal{G}^{\text{pruned}}$ be the FAM after pruning. Then, $\mathcal{G}^{\text{pruned}}$ has one fewer nodes than $\mathcal{G}$, while the associated Full-Sum remains unchanged:

$$\text{FS}(\mathcal{G}) = \text{FS}(\mathcal{G}^{\text{pruned}}).$$

Lemma 3 justifies that pruning reduces the number of summation layers while keeping the associated Full-Sum unchanged. Furthermore, once the edge orientations have been aligned, the updating rules in (7)-(8) only use three operations: multiplication, entry-wise product, and diagonal extraction. These are exactly the operations allowed in SEA terms; see Definition 1.

We now propose the following algorithm; see Algorithm 2.

Algorithm 2 The pruning process.

Input: an induced FAM $\mathcal{G}_{m,k,t}$ from Algorithm 1, with arbitrarily assigned edge orientations.

- a. Check whether the FAM contains any pendant nodes. If not, terminate the algorithm. Otherwise, select an arbitrary pendant node and remove it together with all edges connecting it to its hinge node(s).
- b. Update the node and edge labels using either (7) or (8), with the edge orientations updated as described above, depending on whether the pendant node is of Type I or Type II.
- c. Repeat the above two steps, until either only one node remains or no pendant node exists.

Output: the resultant FAM $\mathcal{G}_{m,k,t}^{\text{final}}$ when the algorithm terminates.

We make a few observations. First, because $\mathcal{G}_{m,k,t}$ has k nodes, and each pruning step reduces the number of nodes by one, Algorithm 2 must terminate in at most $(k - 1)$ steps. Second, by Lemma 3, $\text{FS}(\mathcal{G}_{m,k,t}) = \text{FS}(\mathcal{G}_{m,k,t}^{\text{final}})$. As a result, we can replace $\text{FS}(\mathcal{G}_{m,k,t})$ by $\text{FS}(\mathcal{G}_{m,k,t}^{\text{final}})$ in the decomposition of C_m presented in Theorem 1. Finally, we consider how to compute $\text{FS}(\mathcal{G}_{m,k,t}^{\text{final}})$. When the algorithm terminates, if there is only one node left, $\text{FS}(\mathcal{G}_{m,k,t}^{\text{final}}) = \mathbf{1}'_n v$, where $v \in \mathbb{R}^n$ is the label of this retaining node. By (7)-(8), v is obtained from A and $\mathbf{1}_n$ by using only three operations: multiplication, entry-wise product, and diagonal extraction; therefore, $\mathbf{1}'_n v$ satisfies the requirement of an SEA term (Definition 1). If the algorithm terminates with $\ell > 1$ nodes, then $\text{FS}(\mathcal{G}_{m,k,t}^{\text{final}})$ is an ℓ -layer Full-Sum. The termination criteria requires that no pendant exists, which means that the number of summation layers cannot be further reduced by applying the pruning rules in (7)-(8).

The above observations are formalized in the following theorem:

Theorem 4. Fix $m \geq 3$. Let $b_{m,k}$, $\mathcal{G}_{m,k,t}$ and $a_{m,k,t}$ be the same as in Theorem 1, and let $\mathcal{G}_{m,k,t}^{\text{final}}$ be the output by applying Algorithm 2 to $\mathcal{G}_{m,k,t}$. Then,

$$C_m = \sum_{k=2}^m \sum_{t=1}^{b_{m,k}} a_{m,k,t} \cdot \text{FS}(\mathcal{G}_{m,k,t}^{\text{final}}).$$

Furthermore, there are only two possible cases for each $\mathcal{G}_{m,k,t}^{\text{final}}$:

- Case 1: there is only one node in this FAM. Let $v \in \mathbb{R}^n$ denote the label of this node. Then, $\text{FS}(\mathcal{G}_{m,k,t}^{\text{final}}) = \mathbf{1}'_n v$, and it is an SEA term.
- Case 2: there is no pendant node. Then, there exist at least 4 nodes in this FAM, each with at least 3 immediate neighbors. In addition, $\text{FS}(\mathcal{G}_{m,k,t}^{\text{final}})$ is an ℓ -layer FS term with $4 \leq \ell \leq \lfloor m/2 \rfloor$.

Comparing Theorem 4 with Definition 4, we see that the above decomposition of C_m satisfies the requirement of CEEF for every $m \geq 3$. In this sense, we have successfully solved the CEEF problem.

However, at a practical level, for each specific m , one must still execute Algorithms 1-2 to derive the explicit expression of each $\text{FS}(\mathcal{G}_{m,k,t}^{\text{final}})$ (such as those shown in Examples 1-2). Although this process remains tedious, it no longer requires new theoretical insights; rather, it only requires a research assistant to correctly understand and follow Algorithms 1-2 step by step. For tasks of this nature, AI has important advantages over humans. In the next section, Section 3, we describe how AI can be used to automatically implement the algorithm.

Example 5. We use the FAM in the third row of Table 2 to illustrate the pruning process. This FAM is visualized in the left panel of Figure 5. Its associated FS term is $\sum_{j_1, j_2, j_3} (A \circ A)_{j_1 j_2} (A \circ A)_{j_2 j_3}$. We first prune j_3 and then prune j_1 . The output is a one-node graph with node label $v = (A \circ A)^2 \mathbf{1}_n$. The original FS term has been converted equivalently to $\mathbf{1}'_n v = \mathbf{1}'_n (A \circ A)^2 \mathbf{1}_n$, which is clearly an SEA term.

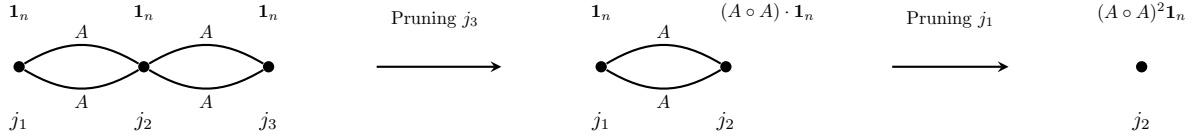


Figure 5: An illustration of the pruning process for a FAM with three nodes. In this example, it happens that edge-label matrices of the original, intermediate, and final FAMs are all symmetric. Thus, edge orientations do not matter and are omitted in the plots.

Our results are novel and nontrivial. We are the first to introduce the notions of SEA terms and IFS terms, which provide a rigorous framework for formulating the CEEF problem. Moreover, the pruning process proposed in this work has not appeared in the existing literature. We also formulate the entire algorithm exclusively in the language of graph theory, making it naturally interpretable and teachable to AI systems.

3 Implementation with AI: Prompts, Results, Validation, and Comparison

In Section 2, we have proposed an two-stage algorithm (Algorithms 1-2) for deriving the CEEF formula of C_m for each given m . However, the execution is labor-extensive. For example, the formula for C_{12} has 1900 terms, and to derive the explicit expression of each individual term is challenging for humans.

To overcome the challenge, we use AI, but (not surprisingly) a *straightforward use* of AI won't work. In particular, we have conducted experiments where we ask large language models (LLMs) to either reason in natural language or write a code to derive a formula C_m as desired (the full prompt is given in the Appendix). Such an approach only works for small values of m (e.g., $m = 3, 4, 5$), where correct answers exist in the literature. When $m \geq 6$, the desired formula either is not well-known or does not exist in the literature, and LLMs fail to output a satisfactory result: It often guesses a few terms with incorrect coefficients and lacks a correct reasoning approach. This suggests that AI is unable to solve the CEEF problem independently. We propose a hugAI approach which combines the strengths of human and AI.

3.1 The human-guided AI (hugAI) approach

In this approach, first, we divide the CEEF problem into many steps, and outline a clear strategy for each step; this is mostly done by us (humans), as given in Section 2. Next, for each step, we either design an algorithm or formulate a clear algorithmic idea, and then ask AI to write a code for implementation. Finally, we ask AI to generate executable Python code that takes m as input and outputs the exact formula for C_m in LaTeX format. The three steps outlined here require different levels of intelligence, which we denote as *level-R (Researcher)*, *level-A (Assistant)*, and *level-C (Coder)*, respectively (see Figure 6). It is of interest to assess the level of intelligence exhibited by AI; we report the results below.

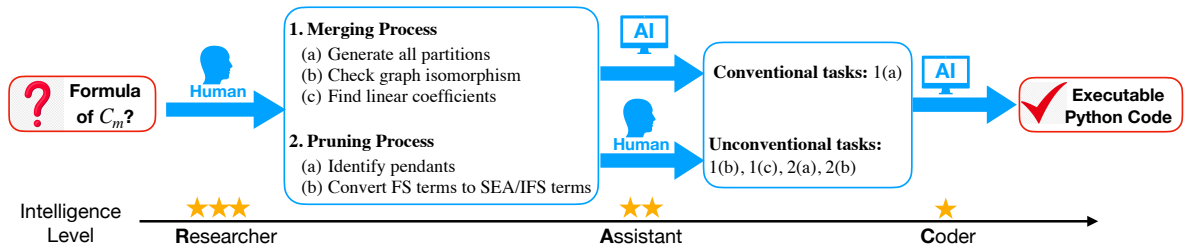


Figure 6: The pipeline of our hugAI approach for counting cycles.

For simplicity, we focus on DeepSeek-R1 (DS-R1), a recent reasoning model that demonstrates competitive performance on benchmark tasks [22]. We also include additional LLMs in our study;

further details are provided in Section 3.4.

First, we prescribe the format for input and output. For input, we teach DS-R1 to express graphs in strings:

An FAM is represented by:

- A dictionary `node_label[v]`, where each node v has a vector label. Initially every node label is `ones_n`.
- A list of oriented edge records `(src, dst, matrix_label)`. The edge contributes `matrix_label[i_src, i_dst]` to the Full Sum.
- Parallel edges are allowed.

For output, we ask DS-R1 to format the output as latex code that prints each term as either a SEA term or an IFS term. The following prompt describes the string format with an example:

If the final graph has at least 4 vertices, denote the vertices count as v , output the graph and the corresponding string in the following form: First, output $\sum_{i_1 i_2 \dots i_v}$. Then, start from the smallest index to the biggest index, for each remaining vertex with `assigned_string_vertex` not being `1_n`, output `(assigned_string_vertex of this vertex)` $\{i_{\{\text{vertex index}\}}\}$. Then, use dictionary order, start from the smallest, for each two vertices with $t \geq 1$ remaining edge between them, output `(assigned_string_edge of edge 1  $\circ$  ...  $\circ$  assigned_string_edge of edge  $t$ )` $\{i_{\{\text{vertex index } 1\}} i_{\{\text{vertex index } 2\}}\}$. Join the above three parts to get the final expression, which should be in the form of a big sum.

Next, we divide our project into five major tasks, three for the merging process (1(a), 1(b), 1(c)), two for the pruning process (2(a), 2(b)), as described in Figure 6 and the caption of Table 5. Based on the difficulty level, 1(a) is a *conventional task* for AI, and others are *unconventional tasks* for AI.

Task 1(a) is well-studied in the literature and AI is able to accomplish it without much struggle. For this task, we only provide a verbal description in the prompt (the full prompt is provided in Section C of the Appendix) and DS-R1 is able to find an appropriate algorithm on its own. This demonstrates that DS-R1 has *level-A (Assistant) intelligence*.

The other 4 tasks are unconventional for AI, and for each of them, AI needs detailed guidance and carefully written prompts. For reasons of space, we only briefly discuss these tasks.

For Task 1(b), DS-R1 does not know what *isomorphism* means in the current setting, so we need to inform it with the precise definition first. After that, DS-R1 knows how to check isomorphism in a brute-force fashion, but the algorithm is very slow. To address the problem, we inform DS-R1 that for checking isomorphism, we only need to consider degree-preserved permutations. This significantly speeds up the computing. Such kind of communication between DS-R1 and us is very common during our study: many times, DS-R1 struggles to understand some concepts, but after several rounds of revision in the prompts, it is able to write a code and produce the right answer.

For Task 1(c), despite that the Möbius function (i.e., the function $\mu(\hat{0}, \sigma_{m,k,t})$ in Theorem 1) is well-known, we must teach DS-R1 how to relate it to our problem. As an example, we inform DS-R1 by prompts (see Section C) that we only need consider the pairs of partitions $\mu(\pi, \sigma)$ with $\pi = \hat{0}$ (the finest partition). With a step-by-step guidance, DS-R1 is able to produce the right results.

For Tasks 2(a)-2(b), since SEA and IFS terms are new concepts, we need to inform DS-R1 with the precise definitions first. Our prompt starts with the prescribed input format (see the text above), followed by detailed instructions on the two types of pruning. The full prompt is contained in the Section C.

In summary, first, DS-R1 is able to accomplish conventional tasks, with correct answers. This demonstrates that DS-R1 has *level-A (Assistant) intelligence*. Second, DS-R1 is able to accomplish unconventional tasks, provided with a step-by-step guidance and carefully written prompts. This demonstrates that DS-R1 has *level-C (Coder) intelligence* for these unconventional tasks: it is able to accomplish the task but requires a detailed step-by-step guidance. What is exciting is that DS-R1 also has preliminary *level-R (Researcher) intelligence*. One example is that, when we initially tried to prove Theorem 1, we didn't realize it was connected to the Möbius function. DS-R1 discovered this connection independently, and we followed its hint to identify the precise linear coefficients in our formula. Another example is, DS-R1 is able to detect inconsistency in our prompt. In one of our prompts, we provided DS-R1 with both a definition and an example of the linear coefficients, but unfortunately, the coefficients in the examples are not consistent with those in the definition. DS-R1 detected such an inconsistency. Using its hint, we revised the prompt, and this time, DS-R1 fully understood our point and provided the correct answer for our question.

3.2 Results

The final outcome of our hugAI approach is a Python code, which takes an integer $m \geq 3$ as input and output the desirable formula as in Theorem 1 and Theorem 4, where each FS term is converted to either a SEA term or an IFS term. Running this code for a given m permits us to obtain the CEEF for C_m .

Table 3: Term count for the combinatorial formula of C_m for $m = 3, \dots, 12$.

m	3	4	5	6	7	8	9	10	11	12
#Terms (a_m)	1	3	3	10	15	44	89	254	633	1900
#SEA Terms	1	3	3	10	15	43	86	239	581	1666
# $O(n^4)$ IFS Terms						1	3	13	46	190
# $O(n^5)$ IFS Terms								2	6	37
# $O(n^6)$ IFS Terms										7

In Section D of the Appendix, we present the resultant formulas of C_m by running this code for $m = 3, 4, \dots, 12$. Table 3 presents the time complexity of computing C_m using such a formula. Note that the time complexity to compute each SEA term is dominated by the cost of calculating matrix multiplications. The time complexity of each IFS term is $O(n^\ell)$, where ℓ is the number of summation layers, i.e. the number of nodes left after the final pruning process. IFS terms only emerge when $m \geq 8$. In this table, we have grouped the IFS terms according to their calculation time complexity.

Remark 4 (CEEF-inspired approximate formulas). These CEEF results are valuable not only for the direct computation of C_m , but also for motivating new approximation methods. For example, in ongoing work, [30] develop an approximate formula for each C_m with $m \geq 8$ whose computational complexity is only $O(n^3)$. The key idea is to approximate each IFS term by a linear combination of SEA terms, yielding an approximation whose error is asymptotically negligible under the random spiked Wigner model. Simulation results show that this CEEF-inspired approximation is both more accurate and computationally more efficient than existing approximate cycle-counting algorithms, such as color coding.

3.3 Validation of the results

We now validate the correctness of these formulas. As established in Section 2, our algorithm (i.e., the high-level reasoning and derivation strategy) has already been rigorously justified; as a result, any correct execution of this strategy is guaranteed to yield a valid formula for C_m . Therefore, the purpose of this validation is not to re-examine the strategy itself, but rather to assess whether the AI correctly understands and executes the prescribed procedure.

Manual validation For $m \leq 8$, the formulas contain relatively few terms (see Table 3) and can be derived manually. We therefore use these hand-derived expressions as the ground truth and find that the formulas produced by our method are correct for all $m \leq 8$. For larger values of m , there are few known results in the literature, except when A is binary (e.g., [41]). In this setting, the formulas are much simpler, as many terms are identical and can be combined (for example, $A \circ \dots \circ A = A$). We apply our method and simplify the resulting expressions accordingly, and find that they agree with those reported in [41] for $m \leq 13$.

Numerical validation Manual validation becomes intractable for large m or the general (non-binary) case. To address this limitation, we introduce a numerical validation pipeline that enables verification of the formulas for both large m and the general case. In this approach, we generate many $n \times n$ matrices A , compute the true C_m , and compare them with the values based on our formulas.

One question is how to compute the true value of C_m . We may search $\binom{n}{m}$ indices in a brute-forth way, but a better solution is to use dynamic programming (DP); see the Supplementary Material for details. The DP method is guaranteed to output the true C_m , but its complexity is $O(n^{2^n})$ for a fixed m . As a consequence, the DP-benchmark is feasible to compute only for small n . We thereby restrict our experiments to $n \leq 16$.

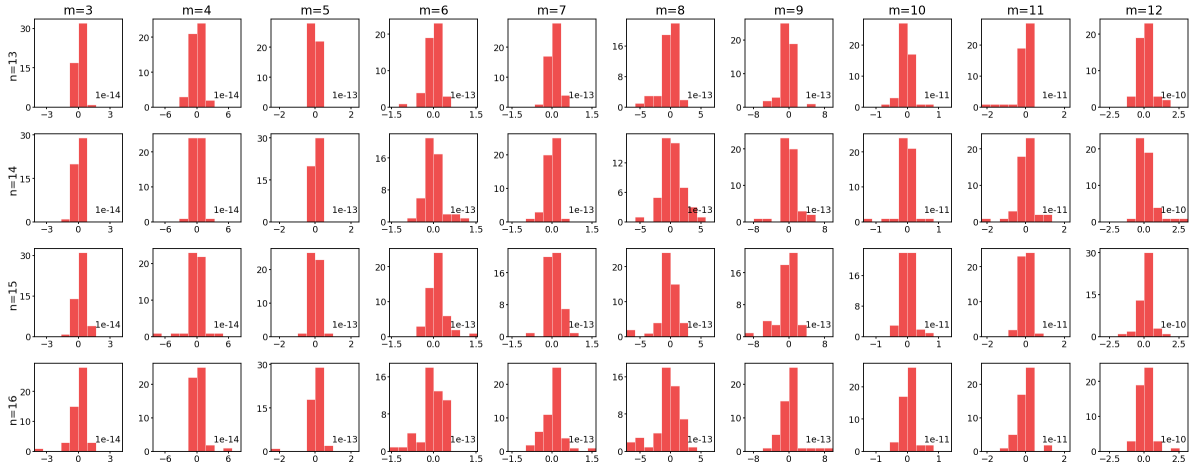


Figure 7: Numerical validation (the histogram of discrepancies between the values by our formulas and the DP-benchmarks in the scale of 10^{-13} or 10^{-14} , where $n = 13, 14, 15, 16$ and $m = 3, \dots, 12$).

Table 4: Average runtime (in seconds) of each C_m for different values of n . Since this experiment is to validate our formula using the computationally expensive DP algorithm, we only consider small values of n . Experiments with much larger values of n are deferred to Section 4.

m	$n = 13$		$n = 14$		$n = 15$		$n = 16$	
	Our Approach	DP	Our Approach	DP	Our Approach	DP	Our Approach	DP
3	2.84×10^{-4}	2.30×10^{-2}	2.93×10^{-4}	3.89×10^{-2}	1.61×10^{-4}	8.14×10^{-2}	1.87×10^{-4}	1.70×10^{-1}
4	1.41×10^{-4}	3.97×10^{-2}	2.51×10^{-4}	5.77×10^{-2}	1.04×10^{-4}	1.07×10^{-1}	1.18×10^{-4}	2.04×10^{-1}
5	1.05×10^{-4}	1.03×10^{-1}	2.30×10^{-4}	1.33×10^{-1}	8.08×10^{-5}	2.21×10^{-1}	8.75×10^{-5}	3.75×10^{-1}
6	3.01×10^{-4}	2.50×10^{-1}	3.68×10^{-4}	3.32×10^{-1}	2.23×10^{-4}	5.56×10^{-1}	2.34×10^{-4}	9.25×10^{-1}
7	4.43×10^{-4}	4.86×10^{-1}	4.76×10^{-4}	6.95×10^{-1}	3.38×10^{-4}	1.24×10^0	3.44×10^{-4}	2.18×10^0
8	4.13×10^{-3}	7.48×10^{-1}	3.75×10^{-3}	1.17×10^0	3.98×10^{-3}	2.27×10^0	3.96×10^{-3}	4.33×10^0
9	1.14×10^{-2}	9.47×10^{-1}	9.40×10^{-3}	1.62×10^0	9.18×10^{-3}	3.42×10^0	9.50×10^{-3}	7.09×10^0
10	4.49×10^{-2}	1.06×10^0	4.04×10^{-2}	1.92×10^0	3.77×10^{-2}	4.35×10^0	4.94×10^{-2}	9.73×10^0
11	1.26×10^{-1}	1.08×10^0	1.21×10^{-1}	2.06×10^0	1.22×10^{-1}	4.88×10^0	1.23×10^{-1}	1.15×10^1
12	5.13×10^{-1}	1.11×10^0	4.95×10^{-1}	2.10×10^0	5.01×10^{-1}	5.10×10^0	5.02×10^{-1}	1.26×10^1

Another question is the following: if we observe matching C_m values across a large number of small-size matrices A , how can we conclude that this provides evidence for the correctness of the formulas? An important observation is that the space of all possible formulas produced by our method is *finite* (see Definitions 1-2 and note that the coefficients in our formulas can only take integer values), whereas the data matrices A we generate take *continuous* values. As a result, the probability that an incorrect formula generates correctly matching values is exactly zero, as formally established in the following lemma:

Lemma 5. Fix $m \geq 3$. Let $C_m^{\text{DS}}(A)$ be the CEEF of $C_m(A)$ produced by the Python code output by DS-R1. If $C_m^{\text{DS}}(A)$ is not algebraically equivalent to $C_m(A)$, then for any $n \geq 2$ and random matrix $A \in \mathbb{R}^{n \times n}$ whose entries have a jointly continuous distribution, $\mathbb{P}(C_m^{\text{DS}}(A) = C_m(A)) = 0$.

In our experiments, we sample A from the Gaussian Orthogonal Ensemble (GOE), where $A_{ij} = A_{ji} \sim \mathcal{N}(0, 1/n)$ for $i < j$ and $A_{ii} \sim \mathcal{N}(0, 2/n)$; and for each setting, we use $t = 50$ samples of A with a fixed random seed. An important issue is the numerical precision required to declare two values as “matching.” For large m , individual terms in the formula may be small in magnitude, so that discrepancies can become effectively indistinguishable at finite precision. Fortunately, when A is generated from the GOE, we can characterize the asymptotic order of each term in the formula, allowing us to calibrate an appropriate tolerance used in validation. Figure 7 presents the histograms of discrepancies between values from our formulas and the DP-benchmark, for a set of (n, m) . We observe that the values are well-matched up to tolerable numerical precision, confirming the correctness of our formulas.

This small-scale experiment also supports the computational value of having a CEEF formula. In Table 4, we report the runtime: For small n , our method is already substantially faster than DP. Note

that for large n , DP is infeasible to implement, while our method is still implementable.

3.4 Comparison of representative LLMs

While our main experiments focus on DS-R1, it is also of interest to evaluate how other LLMs perform under the same prompts. To this end, we test 11 additional models from seven providers (see Table 6 in the Appendix for model details). For all models, web search is disabled, and reasoning (or thinking) mode is enabled whenever supported by the model interface.

Recall that the AI implementation of our method comprises two stages, Merging and Pruning, which together involve five tasks (1a-1c and 2a-2b; see the caption of Table 5x). To facilitate a more systematic comparison across LLMs, we decompose the prompting-and-response pipeline into two components, rather than requiring a single monolithic response:

- Stage 1 (Merging). An LLM receives Stage-1-User-Prompt (see Section C) and is asked to produce a Python function `step1(m)` that enumerates all partitions of the m -cycle, filters self-loops, groups the resulting multi-graphs by isomorphism, and computes the aggregated Möbius coefficient for each canonical type.
- Stage 2 (Pruning). An LLM receives Stage 2 User Prompt (see Section C) together with its own verified Stage 1 code as context. It is asked to produce a function `C_m(A, m)` that applies pendant pruning to each canonical type and constructs the corresponding SEA or IFS terms.

This two-component design avoids a potential output-length truncation issue: a single prompt frequently yields an output that exceeds the 32 k-token limit before completing all tasks. It also enables a better diagnosis—when the formula is incorrect, we can identify whether it is from Stage 1 or Stage 2.

The Stage 1 performance is validated by comparing the canonical multi-graphs and their coefficients with the Stage 1 output of DS-R1, which has been validated extensively in Section 3.3 and is regarded the ground truth. The Stage 2 performance is validated similarly as in Section 3.3, where we apply the Python function `C_m(A, m)` to continuous-valued random matrices and compare them with the DP benchmarks (with a numeric tolerance of 10^{-8}). The results are summarized in Table 5.

Table 5: Evaluation of 12 LLMs (DS-R1 and 11 other models) for the two-stage implementation task. Stage 1 consists of four steps: **Syntax(1)** (code runs), **1a** (valid partition enumeration), **1b** (canonical graph types), **1c** (Möbius coefficients). Stage 2 consists of two steps: **Syntax(2)** (code runs) and **2a/2b** (the generated formula matches the numerical benchmark). A ✓ indicates success throughout the reported validation range; ✗ indicates at least one failure.

Model	Stage 1: Merging				Stage 2: Pruning		Summary
	Syntax(1)	1a	1b	1c	Syntax(2)	2a/2b	
Claude Opus 4.8	✓	✓	✓	✓	✓	✓	Successful
Claude Sonnet 4.6	✓	✓	✓	✓	✓	✓	
DeepSeek R1	✓	✓	✓	✓	✓	✓	
DeepSeek V4 Pro	✓	✓	✓	✓	✓	✓	
GPT-5.5	✓	✓	✓	✓	✓	✓	
Gemini 3.1 Pro	✓	✓	✓	✓	✓	✓	
Gemini 3.5 Flash	✓	✓	✓	✓	✓	✓	
GLM 5.1	✓	✓	✓	✓	✓	✓	
Kimi K2.6	✓	✓	✓	✓	✓	✓	
MiniMax-M3	✓	✓	✓	✓	✓	✓	
MiniMax-M2.7	✓	✓	✓	✓	✗	✗	Stage 1 only
DeepSeek V3.2	✗	✗	✗	✗	✗	✗	Unsuccessful

We have some findings. First, ten of the eleven models successfully complete the Stage 1 tasks. For example, these models correctly identify 41 surviving partitions and 10 canonical multigraphs at $m = 6$, and 715 surviving partitions and 44 canonical multigraphs at $m = 8$. This suggests that the

combinatorial enumeration task, while nontrivial, lies within the reliable capabilities of most frontier models when provided with a precise algorithmic specification. Second, Stage 2 further differentiates model performance. Nine of the models that pass Stage 1 also produce numerically correct formulas over the reported validation range. MiniMax-M2.7 passes the Stage 1 structural checks, but its Stage 2 code cannot be validated because the generated program fails to import. DeepSeek V3.2 fails the Stage 1 structural checks, and its Stage 2 output likewise fails the numerical validation. This pattern highlights the value of separating the Merging and Pruning stages: a failure in Stage 2 has a different interpretation depending on whether the Stage 1 output has first been verified.

Remark 5 (Variability across LLM runs). We observe some variability in LLM performance across repeated runs. For example, GPT-5.5 occasionally fails in Stage 2, whereas DeepSeek V3.2 occasionally succeeds in both stages. The results reported in Table 5 correspond to the majority outcome across multiple runs of each model.

4 Applications

In this section, we present several applications of the cycle count statistic C_m , and illustrate the advantage of using a large value of m . Since these applications require large values of m , solving the CEEF problem is particularly important.

4.1 Detection of weak spikes using cycle count statistics

The spiked Wigner models [10, 13, 14, 19, 33, 42] are widely used in random matrix theory, applied mathematics, statistics, and information theory. Specifically, let $Z \in \mathbb{R}^{n \times n}$ be a symmetric random matrix comprising independent variables in its upper triangle, where $Z_{ij} \sim \mathcal{N}(0, 1/n)$ when $i < j$ and $Z_{ii} \sim \mathcal{N}(0, 2/n)$. In the literature, Z is called a standard Wigner matrix or a Gaussian Orthogonal Ensemble (GOE). The data matrix A satisfies that

$$A = \Omega + Z = \text{‘signal’} + \text{‘noise’}, \quad \text{where } \Omega \text{ has only } K \text{ nonzero eigenvalues } \lambda_1, \dots, \lambda_K. \quad (9)$$

Each λ_k is called a spiked eigenvalue. In the spike detection problem, we test between two hypotheses:

$$H_0 : K = 0 \quad \text{versus} \quad H_1 : K \geq 1.$$

In this framework, [28] showed that $C_m \approx \mathcal{N}(0, 2m)$ when H_0 holds, and C_m is substantially nonzero when H_1 holds (with mild regularity conditions). Therefore, for each m , we may use $C_m/\sqrt{2m}$ as the test statistic. [28] also found that when spiked eigenvalues are only slightly larger than 1 (i.e., weak spikes), the testing power increases rapidly with the order of the cycle count statistic. They suggested to use a weighted linear combination of C_m as the test statistic: For an integer $N \geq 3$, let

$$\phi_N = \frac{1}{\sqrt{B_N}} \sum_{m=1}^N \frac{C_m}{2m}, \quad \text{with} \quad B_N = \sum_{m=1}^N \frac{1}{2m}. \quad (10)$$

Here, C_m denotes the cycle count statistic when $m \geq 3$. For $m < 3$, they defined $C_1 = \text{trace}(A)$, and $C_2 = \text{trace}(A^2) - (n - 1)$. [28] showed that, in the extremely weak spike regime, $N = \Omega(\log(n))$ is necessary for fully distinguishing the two hypotheses. Their work not only demonstrates the effectiveness of cycle count statistics for weak spike detection, but also suggests that incorporating high-order C_m leads to improved statistical performance.

To this end, we conduct a simulation study with $K = 2$ and $n \in \{500, 1000\}$. The two spiked eigenvalues are $\lambda_1 = \lambda_2 = 1$. We sample 100 data matrices A from the null hypothesis and 100 from the alternative hypothesis. The ideal testing error is the smallest sum of type-I and type-II errors on these 200 samples, when the rejection threshold is optimized. In the left panel of Figure 8, we report the ideal testing error of ϕ_m for $m = 1, 2, \dots, 12$, and compare it with the empirical eigenvalues (EE) approach which uses $|\lambda_1|$ as the test statistic. In the right panel of Figure 8, we report the histograms of C_m and ϕ_m under two hypotheses, as well as the difference of means under two hypotheses (Contrast Mean), for $m \in \{3, 6, 9, 10\}$. All the cycle count statistics C_m involved in this experiment are computed using the formulas output by our approach in Sections 2-3.

The results suggest that the cycle-count (CC) testing approach substantially outperforms the EE approach, and the testing error of the CC approach decays evidently as m increases, confirming the

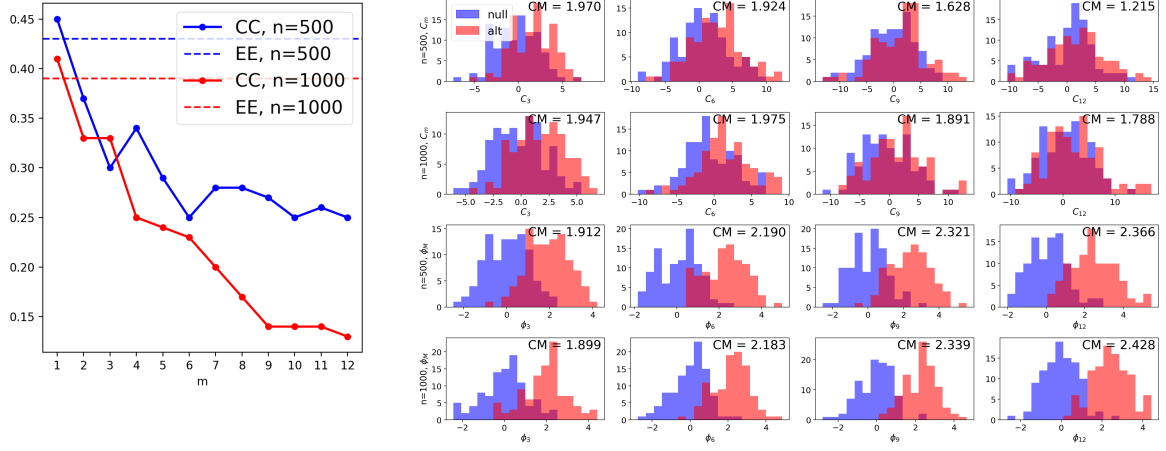


Figure 8: Detection of weak spikes using cycle count statistics. Left: ideal testing error of ϕ_m versus m (the test statistic ϕ_m is a weighted linear combination of cycle count statistics until order m), with a comparison to the testing approach by using empirical eigenvalues (EE). Right: histograms of C_m and ϕ_m under the null and alternative hypotheses, for $m \in \{3, 6, 9, 12\}$, where the numbers on the top right of these plots are the Contrast Mean (CM).

benefit of computing high-order m in this application. When $n = 500$, the brute-forth approach or the dynamic programming is only practically feasible for $m \leq 4$. In contrast, our CEEF formula enables computation for all $m \leq 12$. For example, it takes only a few seconds to compute C_{12} . Even when n increases to 1000, the computing time of C_{12} is still less than a minute.

4.2 Estimation of weak spikes using cycle count statistics

Cycle count statistics are useful not only for detecting spikes but also for estimating them. As shown in [29], C_m provides an estimate of the m th power sum of the spiked eigenvalues, because

$$\mathbb{E}[C_m] \approx \text{trace}(\Omega^m) = \lambda_1^m + \lambda_2^m + \dots + \lambda_K^m := c_m. \quad (11)$$

The problem of estimating the spiked eigenvalues reduces to recovering K numbers from their power sums. Newton's identities [40] and Prony's method [15] are classical approaches to this problem. For convenience, we combine them to a single matrix identity as follows: Let $f(x) = \prod_{k=1}^K (x - \lambda_k) = x^K + \prod_{j=0}^{K-1} a_j x^j$ be the characteristic polynomial associated with $\lambda_1, \dots, \lambda_K$. Write $a = (a_0, a_1, \dots, a_{K-1})^\top$ and $c_m = \sum_{k=1}^K \lambda_k^m$. For any $N \geq 1$, a satisfies the following linear equation:

$$\begin{bmatrix} 0 & 0 & \cdots & 0 & 1 \\ 0 & 0 & \cdots & 2 & c_1 \\ \vdots & \vdots & \ddots & \vdots & \vdots \\ K & c_1 & \cdots & c_{K-2} & c_{K-1} \\ c_1 & c_2 & \cdots & c_{K-1} & c_K \\ \vdots & \vdots & \vdots & \vdots & \vdots \\ c_{N-K} & c_{N-K+1} & \cdots & c_{N-2} & c_{N-1} \end{bmatrix} a = - \begin{bmatrix} c_1 \\ c_2 \\ \vdots \\ c_K \\ c_{K+1} \\ \vdots \\ c_N \end{bmatrix}. \quad (12)$$

Combining the insights in (11)-(12), [29] proposed the Newton-Prony (NP) method for estimating $\lambda_1, \dots, \lambda_K$. This method first estimates c_m by the corresponding cycle count statistic, then solves $\hat{a} = (\hat{a}_0, \hat{a}_1, \dots, \hat{a}_{K-1})^\top$ from the above linear equation, and finally obtains $\hat{\lambda}_1, \dots, \hat{\lambda}_K$ from solving $x^K + \prod_{j=0}^{K-1} \hat{a}_j x^j = 0$.

We conduct a simulation study and compare this NP estimator with the empirical eigenvalue (EE) estimator. We fix $n = 1000$ and $K = 2$, and generate data from the model in (9). The left panel of Figure 9 reports the normalized mean-squared error $\text{NMSE} = [\sum_{k=1}^K (\hat{\lambda}_k - \lambda_k)^2] / (\sum_{k=1}^K \lambda_k^2)$ based on 100 repetitions, for $m = 2, 3, \dots, 12$. The results suggest that the cycle-count-based NP estimator substantially outperforms the empirical eigenvalue estimator. Moreover, the estimation error decreases rapidly as m increases. The right panel of Figure 9 further breaks down the performance by individual eigenvalue, showing that the estimation error decreases with m for both λ_1 and λ_2 .

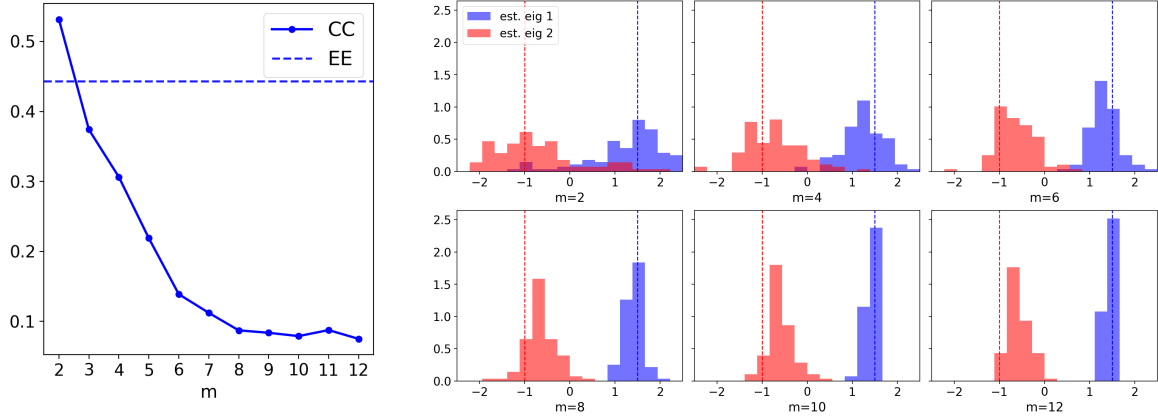


Figure 9: Estimation of weak spikes using cycle count statistics ($n = 1000$, $\lambda_1 = 1.5$, and $\lambda_2 = -1$). Left: normalized mean-squared error (NMSE) versus m . Right: histograms of $\hat{\lambda}_1$ and $\hat{\lambda}_2$ for $m \in \{2, 4, 6, 8, 10, 12\}$.

This experiment confirms that cycle count statistics are effective for estimating spiked eigenvalues, and the estimation accuracy improves as m increases, motivating the computation of high-order C_m . As noted in Section 4.1, our CEEF formula enables the efficient computation of C_m for $m \leq 12$, whereas the brute-force approach is feasible only for $m \leq 4$.

4.3 Network pairwise comparison using cycle count statistics

We consider an application of cycle count statistics on network data. Let $A \in \mathbb{R}^{n \times n}$ be the adjacency matrix of an undirected network. We assume there is no self-loop, so all diagonal entries of A are zero. Suppose the edges are independently generated, and let $\Omega \in \mathbb{R}^{n \times n}$ be a symmetric low-rank matrix such that $\Omega_{ij} = \mathbb{P}(A_{ij} = 1)$. It follows that

$$A = \Omega - \text{diag}(\Omega) + W = \text{'main signal'} - \text{'secondary signal'} + \text{'noise'}, \quad \text{where } W = A - \mathbb{E}[A]. \quad (13)$$

Here, W is a generalized Wigner matrix (compared to the standard Wigner matrix Z in (9), the entries of W are non-Gaussian and can have unequal variances). We call (13) the low-rank network model. It includes well-known models such as the stochastic block model, degree-corrected stochastic block model, and random dot product model as special cases.

Network pairwise comparison [20] has various applications in multiple-network analysis, such as change-point detection [52] for dynamic networks. This problem is formulated as follows: Given two adjacency matrices A_1 and A_2 on the same set of nodes, suppose they both satisfy the low-rank network model in (13), with parameter matrices Ω_1 and Ω_2 , respectively. The aim is to test between two hypotheses:

$$H_0 : \Omega_1 = \Omega_2 \quad \text{versus} \quad H_1 : \Omega_1 \neq \Omega_2.$$

In this framework, cycle count statistics can be computed from A_1 , A_2 , or their difference $\Delta = A_1 - A_2$. We use $C_m(X)$ to denote the cycle count statistic computed from a data matrix X . The IBM test statistic proposed in [27] is given by

$$\psi_m := \frac{C_m(\Delta)}{\sqrt{2^m m [C_m(A_1) + C_m(A_2)]}}. \quad (14)$$

Here, $C_m(\Delta)$ provides an estimate of $\text{trace}((\Omega_1 - \Omega_2)^m)$, which contains the signal for distinguishing the two hypotheses. The quantities $C_m(A_1)$ and $C_m(A_2)$ are combined to estimate $\text{Var}(C_m(\Delta))$, thereby normalizing the test statistic.

To avoid signal cancellation caused by positive and negative eigenvalues of $\Omega_1 - \Omega_2$, [27] recommended using even values of m . Accordingly, we write $U_r = \phi_{2r}$ for all $r \geq 2$. The studies in [27] were restricted to $r \in \{2, 3\}$, where it was noted that larger values of r could potentially improve the testing power but at the cost of substantially heavier computation. Our CEEF formula enables

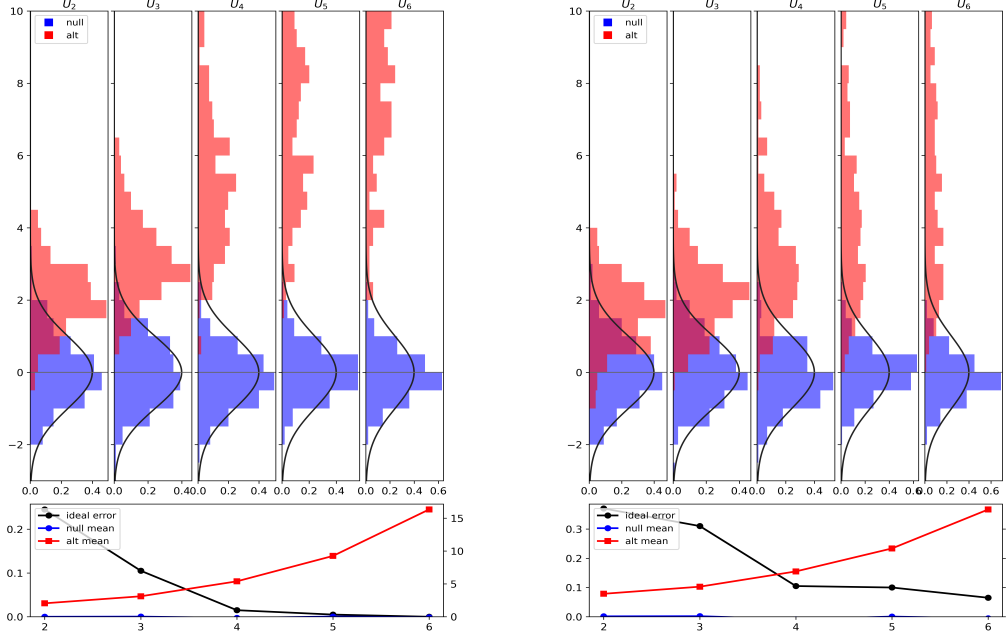


Figure 10: Network pairwise comparison using cycle count statistics. Left: target SNR = 2. Right: target SNR = 1.5.

the computation of U_r for larger values of r . To this end, we conduct a simulation study to examine whether increasing r yields improved performance.

Fix $(n, K) = (500, 2)$. We sample (A_1, A_2) from the degree-corrected mixed membership (DCMM) model [26] with K communities. Let $P = (1 - b_n)I_2 + b_n \mathbf{1}_2 \mathbf{1}_2^\top$, for some $b_n \in (0, 1)$, and $\Theta = \text{diag}(\theta_1, \theta_2, \dots, \theta_n)$ be a diagonal matrix consisting of the degree heterogeneity parameters, and $\Pi = [\pi_1, \pi_2, \dots, \pi_n]^\top$ be the membership matrix so that each π_i describes node i 's mixed membership across two communities. The DCMM model is a special low-rank network model with $\Omega = \Theta \Pi \Pi^\top \Theta$. In our experiments, we generate Θ as follows: draw $\theta_i^* \sim \text{Uniform}(2, 3)$ and then set $\theta_i = \beta_n \theta_i^* / \|\theta^*\|_2$, so that $\|\theta\|_2 = \beta_n$, for some $\beta_n > 0$. We also draw $\pi_i \sim \text{Dirichlet}(1.6, 0.4)$ and $\tilde{\pi}_i \sim \text{Dirichlet}(1, 1)$. Under the null hypothesis, A_1 and A_2 are independently drawn from Model (13) with the same $\Omega = \Theta \Pi \Pi^\top \Theta$. Under the alternative hypothesis, A_1 and A_2 are drawn with Ω and $\tilde{\Omega} = \Theta \tilde{\Pi} \tilde{\Pi}^\top \Theta$, respectively. Following [27], we fix $\beta_n = 6$ and select the value of b_n so that the signal-to-noise ratio (SNR) defined in [27] is equal to a pre-specified number. We consider two scenarios: SNR = 2, and SNR = 1.5.

Figure 10 compares the resulting behavior at these two signal levels. In both panels, the upper portion displays the empirical distributions of the standardized statistics $U_2, U_3, \dots, U_6$ under the null and alternative, together with the reference $N(0, 1)$ law, while the lower portion reports the corresponding ideal classification error and the empirical means under the two hypotheses. The same qualitative pattern appears in both settings: after standardization, the null distributions remain concentrated around zero, whereas the alternative distributions move farther and farther to the right as r increases. Correspondingly, the ideal error decreases with the cycle order in both SNR regimes, so the highest reported orders produce the strongest separation. When the target SNR is reduced from 2 to 1.5, the separation becomes weaker and the ideal errors are uniformly larger, but the same monotone improvement with increasing order remains clearly visible. The main takeaway is therefore that higher-order cycle statistics are consistently more informative, even after the overall signal level is reduced.

5 Discussion

We have developed a general framework for deriving Computationally Efficient Equivalent Forms (CEEFs) for cycle count statistics. The resulting solution is built on new combinatorial concepts and techniques, including Fully Annotated Multigraphs (FAMs), Succinct Expressive Algebraic (SEA) terms, Incompressible Full Sum (IFS) terms, and the merging and pruning procedures developed in Section 2. These ideas do not follow directly from existing results in the literature [11, 24, 26, 39, 41]; in

particular, the notions of SEA terms, FAMs, and their associated full sums appear to be new.

A distinctive feature of this work is the way in which the mathematical framework was executed. For a technically demanding problem such as CEEF, current AI systems should not be expected to solve the problem independently through direct prompting. Instead, our human-guided AI (*hugAI*) approach relies on humans to design the high-level reasoning and derivation strategy and decompose the overall task into many well-formulated subproblems. AI is then used to carry out these subproblems through coding, symbolic manipulation, and systematic enumeration. This division of labor is essential: the human contribution provides the theoretical structure, while AI helps execute the labor-intensive steps that would be impractical to complete manually.

This setting differs from many common uses of AI in theoretical research. Often, researchers present an AI system with a well-formulated problem arising from a particular step of a proof, and both the problem statement and the desired output are relatively concise. In the CEEF problem, by contrast, the output itself becomes increasingly large and intricate as the cycle length m grows; for example, the formula for $m = 12$ contains 1900 terms. This makes it unrealistic to expect current frontier AI systems to generate the full answer correctly in a single interaction. Our approach instead uses AI as an active assistant within a human-designed derivation strategy.

Our approach is also distinct from formal proof assistants such as **Lean**. In formal theorem proving, the mathematical ideas are typically developed by humans, while the proof assistant is used to formalize and verify logical correctness. In contrast, our framework uses AI to carry out nontrivial components of the derivation after the overall strategy has been established. Thus, AI contributes not merely to proof checking, but also to the execution of substantial symbolic and combinatorial tasks.

The validation of AI-generated mathematical output remains an important and largely open problem. Existing validation approaches are most effective when the final output or proof is sufficiently concise to permit manual inspection. In our setting, however, the CEEF formulas become too lengthy to verify manually in their entirety. To address this challenge, we developed a systematic validation pipeline that exploits the intrinsic discreteness of the solution space. Specifically, we validate the symbolic CEEF formulas by evaluating them on simulated continuous-valued data, where numerical agreement provides strong evidence for correctness.

Finally, the value of the CEEF problem extends beyond its combinatorial formulation. Cycle count statistics play an important role in network analysis, covariance matrix testing, and low-rank spectral inference. In Section 4, we study three representative applications: spike detection, spiked eigenvalue estimation, and pairwise network comparison. Our simulations show that higher-order cycle count statistics can yield substantial gains in statistical performance. For the data scales considered in our experiments, brute-force enumeration is feasible only for small m , whereas the proposed CEEF formulas enable efficient computation of C_m for m as large as 12. These results highlight the statistical importance of efficient high-order cycle count computation and suggest several directions for future work.

References

- [1] Boris Alexeev, Moe Putterman, Mehtaab Sawhney, Mark Sellke, and Gregory Valiant. Short proofs in combinatorics, probability and number theory ii. *arXiv preprint arXiv:2604.06609*, 2026.
- [2] Alberto Alfarano, François Charton, and Amaury Hayat. Global lyapunov functions: A long-standing open problem in mathematics with symbolic transformers. In *Advances in Neural Information Processing Systems 38*, 2024. Poster.
- [3] Noga Alon, Phuong Dao, Iman Hajirasouliha, Fereydoun Hormozdiari, and S Cenk Sahinalp. Biomolecular network motif counting and discovery by color coding. *Bioinformatics*, 24(13):i241–i249, 2008.
- [4] Debapratim Banerjee and Zongming Ma. Optimal hypothesis testing for stochastic block models with growing degrees. *arXiv preprint arXiv:1705.05305*, 2017.
- [5] Daniil A. Boiko, Robert MacKnight, Ben Kline, and Gabe Gomes. Autonomous chemical research with large language models. *Nature*, 624(7990):570–578, 2023.
- [6] A.J. Bondy and U.S.R. Murty. *Graph Theory with Applications*. Wiley, 1991.

- [7] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel Ziegler, Jeffrey Wu, Clemens Winter, Chris Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. Language models are few-shot learners. In H. Larochelle, M. Ranzato, R. Hassel, M.F. Balcan, and H. Lin, editors, *Advances in Neural Information Processing Systems*, volume 33, pages 1877–1901. Curran Associates, Inc., 2020.
- [8] Sébastien Bubeck, Christian Coester, Ronen Eldan, Timothy Gowers, Yin Tat Lee, Alexandru Lupsasca, Mehtaab Sawhney, Robert Scherrer, Mark Sellke, Brian K Spears, et al. Early science acceleration experiments with gpt-5. *arXiv preprint arXiv:2511.16072*, 2025.
- [9] Sébastien Bubeck, Jian Ding, Ronen Eldan, and Miklós Z Rácz. Testing for high-dimensional geometry in random graphs. *Random Structures & Algorithms*, 49(3):503–532, 2016.
- [10] Mireille Capitaine, Catherine Donati-Martin, and Delphine Féral. The largest eigenvalues of finite rank deformation of large wigner matrices: convergence and nonuniversality of the fluctuations. *The Annals of Probability*, 37(1):1–47, 2009.
- [11] Y.C. Chang and H.L. Fu. The number of 6-cycles in a graph. *Bulletin of the Institute of Combinatorics and its Applications*, 39, 01 2003.
- [12] Song Xi Chen, Li-Xin Zhang, and Ping-Shou Zhong. Tests for high-dimensional covariance matrices. *Journal of the American Statistical Association*, 105(490):810–819, 2010.
- [13] Chen Cheng, Yuting Wei, and Yuxin Chen. Tackling small eigen-gaps: Fine-grained eigenvector estimation and inference under heteroscedastic noise. *IEEE Transactions on Information Theory*, 67(11):7380–7419, 2021.
- [14] Yuejie Chi, Yue M Lu, and Yuxin Chen. Nonconvex optimization meets low-rank matrix factorization: An overview. *IEEE Transactions on Signal Processing*, 67(20):5239–5269, 2019.
- [15] Gaspard Riche de Prony. Essai expérimental et analytique sur les lois de la dilatabilité des fluides élastiques et sur celles de la force expansive de la vapeur de l’eau et de la vapeur de l’alkool, à différentes températures. *Journal de l’École Polytechnique*, 1:24–76, 1795.
- [16] Edgar Dobriban. Solving a research problem in mathematical statistics with ai assistance. *arXiv preprint arXiv:2511.18828*, 2025.
- [17] Iddo Drori, Sarah Zhang, Reece Shuttlesworth, Leonard Tang, Albert Lu, Elizabeth Ke, Kevin Liu, Linda Chen, Sunny Tran, Newman Cheng, Roman Wang, Nikhil Singh, Taylor L. Patti, Jayson Lynch, Avi Shporer, Nakul Verma, Eugene Wu, and Gilbert Strang. A neural network solves, explains, and generates university math problems by program synthesis and few-shot learning at human level. *Proceedings of the National Academy of Sciences*, 119(32):e2123433119, 2022.
- [18] Evgenii Evstafev. Token-hungry, yet precise: Deepseek r1 highlights the need for multi-step reasoning over speed in math. *ArXiv*, 2024.
- [19] Jianqing Fan, Yingying Fan, Xiao Han, and Jinchi Lv. Asymptotic theory of eigenvectors for random matrices with diverging spikes. *Journal of the American Statistical Association*, 117(538):996–1009, 2022.
- [20] Debarghya Ghoshdastidar, Maurilio Gutzeit, Alexandra Carpentier, and Ulrike Von Luxburg. Two-sample hypothesis testing for inhomogeneous random graphs. *The Annals of Statistics*, 48(4):2208–2229, 2020.
- [21] Mourad Gridach, Jay Nanavati, Khaldoun Zine El Abidine, Lenon Mendes, and Christina Mack. Agentic AI for scientific discovery: A survey of progress, challenges, and future directions. *arXiv preprint arXiv:2503.08979*, 2025.

- [22] Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Ruoyu Zhang, Runxin Xu, Qihao Zhu, Shirong Ma, Peiyi Wang, Xiao Bi, et al. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. *arXiv preprint arXiv:2501.12948*, 2025.
- [23] Thomas R Halford and Keith M Chugg. An algorithm for counting short cycles in bipartite graphs. *IEEE Transactions on Information Theory*, 52(1):287–292, 2005.
- [24] F. Harary. *Graph Theory*. Addison-Wesley Series in Mathematics. Addison-Wesley Longman, Incorporated, 1969.
- [25] Carlos E Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *The Twelfth International Conference on Learning Representations (ICLR)*, 2024. Oral presentation.
- [26] Jiashun Jin, Zheng Tracy Ke, and Shengming Luo. Optimal adaptivity of signed-polygon statistics for network testing. *The Annals of Statistics*, 49(6):3408–3433, 2021.
- [27] Jiashun Jin, Zheng Tracy Ke, Shengming Luo, and Yucong Ma. Optimal network pairwise comparison. *Journal of the American Statistical Association*, 120(550):1048–1062, 2025.
- [28] Jiashun Jin, Zheng Tracy Ke, Bingcheng Sui, and Jingming Wang. Detecting weak spikes by counting cycles. Manuscript, 2025.
- [29] Jiashun Jin, Zheng Tracy Ke, Bingcheng Sui, and Jingming Wang. Estimating spikes with newton and prony. Manuscript, 2026.
- [30] Jiashun Jin, Bingcheng Sui, and Jingming Wang. Approximate cycle counts with $O(n^3)$ complexity. *Manuscript*, 2025.
- [31] John Jumper, Richard Evans, Alexander Pritzel, Tim Green, et al. Highly accurate protein structure prediction with AlphaFold. *Nature*, 596(7873):583–589, 2021.
- [32] Mehdi Karimi and Amir H Banihashemi. Message-passing algorithms for counting short cycles in a graph. *IEEE Transactions on Communications*, 61(2):485–495, 2012.
- [33] Antti Knowles and Jun Yin. The outliers of a deformed wigner matrix. *The Annals of Probability*, 42(5):1980–2031, 2014.
- [34] Guillaume Lample, Marie-Anne Lachaux, Thibaut Lavril, Xavier Martinet, Amaury Hayat, Gabriel Ebner, Aurélien Rodriguez, and Timothée Lacroix. Hypertree proof search for neural theorem proving. In *Advances in Neural Information Processing Systems 35*, 2022.
- [35] Andres M. Bran, Sam Cox, Oliver Schilter, Carlo Baldassari, Andrew D White, and Philippe Schwaller. Augmenting large language models with chemistry tools. *Nature Machine Intelligence*, 6(5):525–535, 2024.
- [36] Cheng Mao, Yihong Wu, Jiaming Xu, and Sophie H Yu. Testing network correlation efficiently via counting trees. *The Annals of Statistics*, 52(6):2483–2505, 2024.
- [37] Amil Merchant, Simon Batzner, Samuel S Schoenholz, Muratahan Aykol, Gowoon Cheon, and Ekin Dogus Cubuk. Scaling deep learning for materials discovery. *Nature*, 624(7990):80–85, 2023.
- [38] Ron Milo, Shai Shen-Orr, Shalev Itzkovitz, Nadav Kashtan, Dmitri Chklovskii, and Uri Alon. Network motifs: simple building blocks of complex networks. *Science*, 298(5594):824–827, 2002.
- [39] Nazanin Movarraei and Samina Boxwala. On the number of cycles in a graph. *Open Journal of Discrete Mathematics*, 6:41–49, 03 2016.
- [40] Isaac Newton. *Arithmetica Universalis*. Typis Pearsonianis, London, 1707.
- [41] Sergey Perepechko and Anton Voropaev. The number of fixed length cycles in an undirected graph. explicit formulae in case of small lengths. *Mathematical Modeling and Computational Physics (MMCP2009)*, 148, 2009.

- [42] Amelia Perry, Alexander S Wein, Afonso S Bandeira, and Ankur Moitra. Optimality and sub-optimality of pca i: Spiked random matrix models. *The Annals of Statistics*, 46(5):2416–2451, 2018.
- [43] Yumou Qiu and Song Xi Chen. Test for bandedness of high-dimensional covariance matrices and bandwidth estimation. *The Annals of Statistics*, 40(3):1285–1314, 2012.
- [44] Maziar Raissi, Paris Perdikaris, and George Em Karniadakis. Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations. *Journal of Computational Physics*, 378:686–707, 2019.
- [45] Pedro Ribeiro, Pedro Paredes, Miguel EP Silva, David Aparicio, and Fernando Silva. A survey on subgraph counting: concepts, algorithms, and applications to network motifs and graphlets. *ACM computing surveys (csur)*, 54(2):1–36, 2021.
- [46] Bernardino Romera-Paredes, Mohammadamin Barekatain, Alexander Novikov, Matej Balog, M. Pawan Kumar, Emilien Dupont, Francisco J. R. Ruiz, Jordan S. Ellenberg, Pengming Wang, Omar Fawzi, Pushmeet Kohli, and Alhussein Fawzi. Mathematical discoveries from program search with large language models. *Nature*, 625:468–475, 2024.
- [47] Richard P. Stanley. *Enumerative Combinatorics*. Cambridge Studies in Advanced Mathematics. Cambridge University Press, 2 edition, 2011.
- [48] Trieu H. Trinh and *et al.* Solving olympiad geometry without human demonstrations. *Nature*, 625(7995):4760–482, 2024.
- [49] Lei Wang, Chen Ma, Xueyang Feng, Zeyu Zhang, Hao Yang, Jingsen Zhang, Zhiyuan Chen, Jiakai Tang, Xu Chen, Yankai Lin, et al. A survey on large language model based autonomous agents. *Frontiers of Computer Science*, 18(6):186345, 2024.
- [50] Ziao Wang, Weina Wang, and Lele Wang. Efficient algorithms for attributed graph alignment with vanishing edge correlation. *IEEE Transactions on Information Theory*, 2025.
- [51] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, brian ichter, Fei Xia, Ed Chi, Quoc V Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models. In S. Koyejo, S. Mohamed, A. Agarwal, D. Belgrave, K. Cho, and A. Oh, editors, *Advances in Neural Information Processing Systems*, volume 35, pages 24824–24837. Curran Associates, Inc., 2022.
- [52] Yao Xie and David Siegmund. Sequential multi-sensor change-point detection. *The Annals of Statistics*, 41(2):670–692, 2013.
- [53] Xuan Zhang, Limei Wang, Jacob Helwig, Youzhi Luo, Cong Fu, Yaochen Xie, Meng Liu, Yuchao Lin, Zhao Xu, Keqiang Yan, et al. Artificial intelligence for science in quantum, atomistic, and continuum systems. *Foundations and Trends® in Machine Learning*, 18(4):385–849, 2025.

A Proofs

In this section, we provide complete proofs for all lemmas and theorems presented in the main manuscript.

A.1 Properties of the induced multi-graphs

Recall that $S = \{i_1, i_2, \dots, i_m\}$ and $\mathcal{G}$ is the undirected simple graph whose node set is S and there are edges between $i_1 \& i_2, i_2 \& i_3, \dots, i_m \& i_1$, but nowhere else. We use $\sigma = \{S_1, S_2, \dots, S_k\}$ to denote a general size- k partition on S , where $S_1, S_2, \dots, S_k$ are non-overlapping subsets whose union is equal to S , and k is the size of this partition, for which we often write $|\sigma| = k$. As described in Section 2.2, each partition σ induces a multi-graph $\mathcal{G}_\sigma$: It modifies $\mathcal{G}$ by merging all nodes in each S_j to one, while keeping the edges unchanged (note that $\mathcal{G}_\sigma$ has $|\sigma|$ nodes in this case).

Definition 13. We call $\mathcal{G}_\sigma$ non-trivial if it does not have self-loops, and we call a partition σ non-trivial if $\mathcal{G}_\sigma$ is non-trivial.

The next lemma gives some useful properties for these induced $\mathcal{G}_\sigma$:

Lemma 6. *A non-trivial induced multi-graph $\mathcal{G}_\sigma$ has the following properties: (a) It is connected. (b) Its size satisfies that $k \geq 2$. (c) The degree of each node is even.*

Proof of Lemma 6: For (a), we recall that $\mathcal{G}_\sigma$ is obtained from merging nodes in $\mathcal{G}$. If a graph is connected before the merging process, it will also be connected afterwards. Since the original graph $\mathcal{G}$ is connected, each $\mathcal{G}_\sigma$ must also be connected. For (b), we note that a 1-node multi-graph can only have self-loops, which contradict the definition of a non-trivial multi-graph.

To prove (c), we study how merging two nodes a and b in a multi-graph affects node degrees. Suppose the degree of each node is even before merging. For all nodes other than a and b , their degrees are unchanged during merging. For a and b , let d_a and d_b be their respective degrees before merging, and let w_{ab} be the number of edges between a and b . Here, both d_a and d_b are even, and w_{ab} can be either even or odd. After these two nodes are merged, the degree of the new node $a \& b$ becomes

$$d^* = (d_a - w_{ab}) + (d_b - w_{ab}), \quad \text{which must be an even number.}$$

So far, we have shown the following: If each node has an even degree before merging, the same is true after merging. Therefore, the claim in (c) follows immediately by noting that each node has an even degree in the original graph $\mathcal{G}$. $\square$

A.2 Proof of Theorem 1

Write $S = \{i_1, i_2, \dots, i_m\}$ and let $\Pi(S)$ be the collection of all partitions over S . For any $\sigma \in \Pi(S)$, let $\{j_1, j_2, \dots, j_k\}$ be the nodes in the induced multi-graph $\mathcal{G}_\sigma$, and let $w(a, b)$ be the edge weight between any two nodes j_a and j_b . Let $f_\sigma(A, j_1, j_2, \dots, j_k) := \prod_{1 \leq a < b \leq k} A_{j_a j_b}^{w(a, b)}$. We introduce two quantities:

$$R_\sigma^{\text{DS}} := \sum_{\substack{1 \leq j_1, j_2, \dots, j_k \leq n \\ (\text{distinct})}} f_\sigma(A, j_1, j_2, \dots, j_k), \quad R_\sigma^{\text{FS}} := \sum_{1 \leq j_1, j_2, \dots, j_k \leq n} f_\sigma(A, j_1, j_2, \dots, j_k). \quad (15)$$

We call them the (merged) σ -DS term and σ -FS term, respectively.

Given partitions σ and τ , we use $\tau \preceq \sigma$ to denote that τ is obtained by splitting blocks in σ , and we use $\tau \prec \sigma$ to denote that $\tau \preceq \sigma$ but $\tau \neq \sigma$. Our proof is based on the following key lemma:

Lemma 7. *Let $\mathcal{D}(\sigma)$ be an arbitrary mapping that maps each $\sigma \in \Pi(S)$ to a real value. Then,*

$$\sum_{\sigma \in \Pi(S)} \mathcal{D}(\sigma) R_\sigma^{\text{FS}} = \sum_{\sigma \in \Pi(S)} \mathcal{F}_1(\sigma) R_\sigma^{\text{DS}}, \quad \text{with } \mathcal{F}_1(\sigma) := \sum_{\tau: \tau \preceq \sigma} \mathcal{D}(\tau),$$

and

$$\sum_{\sigma \in \Pi(S)} \mathcal{D}(\sigma) R_\sigma^{\text{DS}} = \sum_{\sigma \in \Pi(S)} \mathcal{F}_2(\sigma) R_\sigma^{\text{FS}}, \quad \text{with } \mathcal{F}_2(\sigma) := \begin{cases} \mathcal{D}(\sigma), & \text{if } \sigma = \hat{0}, \\ \mathcal{D}(\sigma) - \sum_{\tau: \tau \prec \sigma} \mathcal{F}_2(\tau), & \text{if } \sigma \neq \hat{0}. \end{cases}$$

Proof of Lemma 7: Before proving this lemma, we argue that $\mathcal{F}_2(\cdot)$ is well-defined. For each $\sigma \in \Pi(S)$, let $\text{Child}(\sigma) = \{\tau : \tau \prec \sigma\}$. We use Π^* to denote the collection of σ for which $\mathcal{F}_2(\sigma)$ has been defined. This set initially only contains one element, which is $\hat{0}$. Next, for any σ such that $\text{Child}(\sigma) \subset \Pi^*$, we can use the formula to obtain $\mathcal{F}_2(\sigma)$. We then add these σ into Π^* . After Π^* is updated, we search again those σ such that $\text{Child}(\sigma) \subset \Pi^*$, use the formula to obtain $\mathcal{F}_2(\sigma)$, and add them to Π^* . This procedure is repeated until every σ is in Π^* . Therefore, $\mathcal{F}_2(\cdot)$ is well-defined.

Consider the first claim. Let $\tau_0 = \hat{0}$. We introduce a mapping $\mathcal{M}$ that maps each $(i_1, i_2, \dots, i_m) \in \{1, 2, \dots, n\}^m$ to a partition σ , depending on which of these m indices are equal to each other (this mapping has also been used in (26), with a more careful explanation there). We can easily see that

$$R_{\tau_0}^{\text{FS}} = \sum_{1 \leq i_1, \dots, i_m \leq n} f_{\tau_0}(A, i_1, \dots, i_m) = \sum_{\sigma \in \Pi(S)} \sum_{\substack{1 \leq i_1, \dots, i_m \leq n, \\ \mathcal{M}(i_1, \dots, i_m) = \sigma}} f_{\tau_0}(A, i_1, \dots, i_m).$$

Additionally, in (27), we have proved the following result: For each σ , letting $j_1, j_2, \dots, j_k$ denote the nodes in $\mathcal{G}_\sigma$, it holds that

$$\sum_{\substack{1 \leq i_1, \dots, i_m \leq n, \\ \mathcal{M}(i_1, \dots, i_m) = \sigma}} f_{\tau_0}(A, i_1, \dots, i_m) = \sum_{\substack{1 \leq j_1, j_2, \dots, j_k \leq n \\ (\text{distinct})}} f_\sigma(A, j_1, j_2, \dots, j_k) = R_\sigma^{\text{DS}}.$$

Combining the above two equations and noticing that $\tau_0 \preceq \sigma$ for all $\sigma \in \Pi(S)$, we obtain:

$$R_{\tau_0}^{\text{FS}} = \sum_{\sigma \in \Pi(S)} R_\sigma^{\text{DS}} = \sum_{\sigma: \tau_0 \preceq \sigma} R_\sigma^{\text{DS}}, \quad \text{for } \tau_0 = \hat{0}.$$

For $\tau \neq \tau_0$, we can mimic the above proof and consider all partitions over the nodes in $\mathcal{G}_\tau$. We can similarly show that

$$R_\tau^{\text{FS}} = \sum_{\sigma: \tau \preceq \sigma} R_\sigma^{\text{DS}}, \quad \text{for all } \tau \in \Pi(S). \quad (16)$$

It follows by (16) that

$$\sum_{\tau \in \Pi(S)} \mathcal{D}(\tau) R_\tau^{\text{FS}} = \sum_{\tau \in \Pi(S)} \mathcal{D}(\tau) \left(\sum_{\sigma: \tau \preceq \sigma} R_\sigma^{\text{DS}} \right) = \sum_{\sigma \in \Pi(S)} \left[\sum_{\tau: \tau \preceq \sigma} \mathcal{D}(\tau) \right] R_\sigma^{\text{DS}}.$$

This proves the first claim.

Consider the second claim. When there exists a mapping $\mathcal{F}_2(\sigma)$ such that $\sum_{\sigma \in \Pi(S)} \mathcal{F}_2(\sigma) R_\sigma^{\text{FS}} = \sum_{\sigma \in \Pi(S)} \mathcal{D}(\sigma) R_\sigma^{\text{DS}}$, we can apply the first claim of this lemma to conclude that

$$\mathcal{D}(\sigma) = \sum_{\tau: \tau \preceq \sigma} \mathcal{F}_2(\tau), \quad \text{for all } \sigma \in \Pi(S). \quad (17)$$

We can rewrite (17) as $\mathcal{D}(\sigma) = \mathcal{F}_2(\sigma) + \sum_{\tau: \tau \prec \sigma} \mathcal{F}_2(\tau)$, which implies that

$$\mathcal{F}_2(\sigma) = \mathcal{D}(\sigma) - \sum_{\tau: \tau \prec \sigma} \mathcal{F}_2(\tau), \quad \text{for all } \sigma \in \Pi(S).$$

This proves the second claim of Lemma 7. $\square$

We now proceed to prove Theorem 1. In the special case where $\sigma = \hat{0}$ (the finest partition), the induced $\mathcal{G}_\sigma$ has m nodes and a single edge between each of $i_1 \& i_2, i_2 \& i_3, \dots, i_m \& i_1$. It follows that $f_{\hat{0}}(A, j_1, j_2, \dots, j_k) = A_{i_1 i_2} A_{i_2 i_3} \dots A_{i_m i_1}$. As a result,

$$C_m = \sum_{\substack{1 \leq i_1, i_2, \dots, i_m \leq n \\ (\text{distinct})}} f_{\hat{0}}(A, i_1, i_2, \dots, i_m) = R_{\hat{0}}^{\text{DS}}.$$

We can write it equivalently as

$$C_m = \sum_{\sigma \in \Pi(S)} \mathcal{D}(\sigma) R_\sigma^{\text{DS}}, \quad \text{with } \mathcal{D}(\sigma) := \begin{cases} 1 & \text{if } \sigma = \hat{0}, \\ 0 & \text{otherwise.} \end{cases} \quad (18)$$

Using the second claim of Lemma 7, we immediately obtain that

$$C_m = \sum_{\sigma \in \Pi(S)} \mathcal{F}(\sigma) R_\sigma^{\text{FS}}, \quad \text{with } \mathcal{F}(\sigma) = \begin{cases} 1 & \text{if } \sigma = \hat{0}, \\ - \sum_{\hat{0} \preceq \tau \prec \sigma} \mathcal{F}(\tau) & \text{if } \sigma \neq \hat{0}. \end{cases} \quad (19)$$

Definition 14 (The Möbius function). For any $\pi, \sigma \in \Pi(S)$ with $\pi \prec \sigma$, $\mu(\pi, \pi) = 1$, and $\mu(\pi, \sigma)$ is defined recursively by

$$\mu(\pi, \sigma) = - \sum_{\pi \preceq \tau \prec \sigma} \mu(\pi, \tau).$$

Comparing $\mathcal{F}(\sigma)$ in (19) with Definition 14, we conclude that

$$\mathcal{F}(\sigma) = \mu(\hat{0}, \sigma), \quad \text{for } \sigma \in \Pi(S). \quad (20)$$

The Möbius function $\mu(\hat{0}, \sigma)$ has an explicit expression, which is well-known in the literature and presented in the following lemma (e.g., see [47] for a proof):

Lemma 8 (Explicit formula of Möbius functions on partition lattices). *For any partition σ with k blocks $\{S_1, S_2, \dots, S_k\}$ of $\{1, 2, \dots, m\}$, we have $\mu(\hat{0}, \sigma) = (-1)^{m-k} \prod_{i=1}^k (|S_i| - 1)!$.*

We plug (20) into (19) and let $\mathcal{G}_{m,k,t}$ be the same isomorphic class as in (31). It yields that

$$C_m = \sum_{\sigma \in \Pi(S)} \mu(\hat{0}, \sigma) R_\sigma^{\text{FS}} = \sum_{k=2}^m \sum_{t=1}^{b_{m,k}} \sum_{\sigma: \mathcal{G}_\sigma \in \mathcal{G}_{m,k,t}} \mu(\hat{0}, \sigma) R_\sigma^{\text{FS}}, \quad (21)$$

For partitions σ and $\tilde{\sigma}$ such that their corresponding multi-graphs are isomorphic, we have $R_\sigma^{\text{FS}} = R_{\tilde{\sigma}}^{\text{FS}}$ (the derivation for this equality is similar to that for (30), except for a minor difference that the distinct sum there is replaced by the full sum here). Additionally, from the expression in Lemma 8, we also observe that $\mu(\hat{0}, \sigma) = \mu(\hat{0}, \tilde{\sigma})$. Let $\sigma_{m,k,t}$ be the canonical partition in $\mathcal{G}_{m,k,t}$. It is seen that

$$\mu(\hat{0}, \sigma) R_\sigma^{\text{FS}} = \mu(\hat{0}, \sigma_{m,k,t}) R_{\sigma_{m,k,t}}^{\text{FS}}, \quad \text{for all } \sigma \text{ such that } \mathcal{G}_\sigma \in \mathcal{G}_{m,k,t}.$$

Recall that $d_{m,k,t} = |\mathcal{G}_{m,k,t}|$. We plug the above equality into (21) to obtain:

$$C_m = \sum_{k=2}^{m-1} \sum_{t=1}^{b_{m,k}} d_{m,k,t} \cdot \mu(\hat{0}, \sigma_{m,k,t}) \cdot R_{\sigma_{m,k,t}}^{\text{FS}}. \quad (22)$$

Finally, we let $f_{m,k,t}(A, j_1, j_2, \dots, j_k) := f_{\sigma_{m,k,t}}(A, j_1, j_2, \dots, j_k)$ and use the expression of R_σ^{FS} in (20). It follows that

$$R_{\sigma_{m,k,t}}^{\text{FS}} = \sum_{1 \leq j_1, j_2, \dots, j_k \leq n} f_{m,k,t}(A, j_1, j_2, \dots, j_k).$$

Let $a_{m,k,t} := d_{m,k,t} \cdot \mu(\hat{0}, \sigma_{m,k,t})$. We plug the above equality into (22) to get

$$C_m = \sum_{k=2}^{m-1} \sum_{t=1}^{b_{m,k}} a_{m,k,t} \sum_{j_1, j_2, \dots, j_k} f_{m,k,t}(A, j_1, j_2, \dots, j_k). \quad (23)$$

Let $h_{m,k,t} = \prod_{i=1}^k (|S_i| - 1)!$ as defined in (6), with $\{S_1, S_2, \dots, S_k\}$ being the blocks of partition $\sigma_{m,k,t}$. It follows by Lemma 8 that $\mu(\hat{0}, \sigma_{m,k,t}) = (-1)^{m-k} \cdot h_{m,k,t}$. It follows that

$$a_{m,k,t} = d_{m,k,t} \cdot (-1)^{m-k} \cdot h_{m,k,t}. \quad (24)$$

The claim follows by combining (24) with (23). $\square$

A.3 Proof of Corollary 2

Recall that $C_m = \sum_{i_1, \dots, i_m (\text{distinct})} A_{i_1 i_2} A_{i_2 i_3} \dots A_{i_m i_1}$. Without the requirement of having distinct indices, we can easily deduce $\sum_{i_1, \dots, i_m} A_{i_1 i_2} A_{i_2 i_3} \dots A_{i_m i_1} = \text{tr}(A^m)$. As a result,

$$C_m = \text{tr}(A^m) - R_m, \quad \text{with } R_m := \sum_{\substack{1 \leq i_1, i_2, \dots, i_m \leq n \\ (\text{not distinct})}} A_{i_1 i_2} A_{i_2 i_3} \dots A_{i_m i_1}. \quad (25)$$

We study R_m . When $i_1, \dots, i_m$ are non-distinct, there can be $k \leq m-1$ distinct indices. When $k=1$, since the diagonal entries of A are zero, we have $A_{i_1 i_2} A_{i_2 i_3} \dots A_{i_m i_1} = 0$. It suffices to consider $2 \leq k \leq m-1$, and we can write

$$R_m = \sum_{k=2}^{m-1} \sum_{\substack{1 \leq i_1, i_2, \dots, i_m \leq n \\ |\{i_1, i_2, \dots, i_m\}|=k}} A_{i_1 i_2} A_{i_2 i_3} \dots A_{i_m i_1}.$$

When $|\{i_1, i_2, \dots, i_m\}| = k$, these m indices are divided into k non-overlapping sets and correspond to a size- k partition. To this end, we define a mapping $\mathcal{M}$ that maps each $(i_1, i_2, \dots, i_m)$ to a partition σ . Let $\Pi(S)$ denote the collection of all partitions with $2 \leq |\sigma| \leq m-1$. We can further write

$$R_m = \sum_{k=2}^{m-1} \sum_{\sigma \in \Pi(S): |\sigma|=k} \sum_{\substack{1 \leq i_1, i_2, \dots, i_m \leq n \\ \mathcal{M}(i_1, i_2, \dots, i_m) = \sigma}} A_{i_1 i_2} A_{i_2 i_3} \dots A_{i_m i_1}. \quad (26)$$

For each σ with $|\sigma| = k$, let $j_1, j_2, \dots, j_k$ denote the nodes in the corresponding multi-graph $\mathcal{G}_\sigma$. When $\mathcal{M}(i_1, i_2, \dots, i_m) = \sigma$, we map each of $i_1, i_2, \dots, i_m$ to one of $j_1, j_2, \dots, j_k$. For example, when $m = 4$ and $\sigma = \{\{i_1, i_3\}, \{i_2\}, \{i_4\}\}$, the original indices i_1, i_2, i_3, i_4 are mapped to the new indices j_1, j_2, j_1, j_3 , respectively. Consequently,

$$A_{i_1 i_2} A_{i_2 i_3} A_{i_3 i_4} A_{i_4 i_1} = A_{j_1 j_2} A_{j_2 j_1} A_{j_1 j_3} A_{j_3 j_1} = A_{j_1 j_2}^2 A_{j_1 j_3}^2.$$

More generally, let $w(a, b)$ denote the edge weight between nodes j_a and j_b in $\mathcal{G}_\sigma$. We observe that each $A_{j_a j_b}$ is repeated $w(a, b)$ times in $A_{i_1 i_2} A_{i_2 i_3} \dots A_{i_m i_1}$, which implies that

$$A_{i_1 i_2} A_{i_2 i_3} \dots A_{i_m i_1} = \prod_{1 \leq a < b \leq k} A_{j_a j_b}^{w(a, b)}, \quad \text{when } \mathcal{M}(i_1, i_2, \dots, i_m) = \sigma.$$

Furthermore, when we sum over $1 \leq i_1, \dots, i_m \leq n$ under the constraint that $\mathcal{M}(i_1, \dots, i_m) = \sigma$, it is equivalent to summing over $1 \leq j_1, \dots, j_k \leq n$ under the constraint that $j_1, \dots, j_k$ are distinct. Combining the above arguments gives

$$\sum_{\substack{1 \leq i_1, i_2, \dots, i_m \leq n \\ \mathcal{M}(i_1, i_2, \dots, i_m) = \sigma}} A_{i_1 i_2} A_{i_2 i_3} \dots A_{i_m i_1} = \sum_{\substack{1 \leq j_1, j_2, \dots, j_k \leq n \\ (\text{distinct})}} \left(\prod_{1 \leq a < b \leq k} A_{j_a j_b}^{w(a, b)} \right). \quad (27)$$

We plug (27) into (26) to obtain:

$$R_m = \sum_{k=2}^{m-1} \sum_{\sigma \in \Pi(S): |\sigma|=k} \sum_{\substack{1 \leq j_1, j_2, \dots, j_k \leq n \\ (\text{distinct})}} \left(\prod_{1 \leq a < b \leq k} A_{j_a j_b}^{w(a, b)} \right). \quad (28)$$

Write $f_\sigma(A, j_1, j_2, \dots, j_k) := \prod_{1 \leq a < b \leq k} A_{j_a j_b}^{w(a, b)}$ for brevity. We can express (28) equivalently as

$$R_m = \sum_{k=2}^{m-1} \sum_{\sigma \in \Pi(S): |\sigma|=k} R_\sigma, \quad \text{with } R_\sigma := \sum_{\substack{1 \leq j_1, j_2, \dots, j_k \leq n \\ (\text{distinct})}} f_\sigma(A, j_1, j_2, \dots, j_k).$$

We group all multi-graphs $\mathcal{G}_\sigma$ first by size and then by isomorphism. Let $b_{m,k}$ be the total number of isomorphic classes for size- k multi-graphs, and let $\mathcal{G}_{m,k,t}$ denote the t -th isomorphic class, for $1 \leq t \leq b_{m,k}$. It follows that

$$R_m = \sum_{k=2}^{m-1} \sum_{t=1}^{b_{m,k}} \sum_{\sigma: \mathcal{G}_\sigma \in \mathcal{G}_{m,k,t}} R_\sigma. \quad (29)$$

Consider two partitions σ and $\tilde{\sigma}$ such that $\mathcal{G}_\sigma$ and $\mathcal{G}_{\tilde{\sigma}}$ are isomorphic multi-graphs. The definition of isomorphism implies that there exists a permutation $\mathcal{P}$ on $\{j_1, j_2, \dots, j_k\}$ such that

$$f_{\tilde{\sigma}}(A, j_1, j_2, \dots, j_k) = f_\sigma(A, \mathcal{P}(j_1), \mathcal{P}(j_2), \dots, \mathcal{P}(j_k)).$$

When we sum over $1 \leq j_1, \dots, j_k \leq n$ and require them to be distinct, the above permutation does not matter. As a result,

$$R_\sigma = R_{\tilde{\sigma}}, \quad \text{whenever } \mathcal{G}_\sigma \text{ and } \mathcal{G}_{\tilde{\sigma}} \text{ are isomorphic.} \quad (30)$$

To this end, we pick the canonical partition $\sigma_{m,k,t} \in \mathcal{G}_{m,k,t}$ and define $f_{m,k,t}(A, j_1, j_2, \dots, j_k) := f_{\sigma_{m,k,t}}(A, j_1, j_2, \dots, j_k)$. Combining (30) with (29), we obtain:

$$\begin{aligned} R_m &= \sum_{k=2}^{m-1} \sum_{t=1}^{b_{m,k}} d_{m,k,t} \cdot R_{\sigma_{m,k,t}}, \quad \text{where } d_{m,k,t} = |\mathcal{G}_{m,k,t}|, \\ &= \sum_{k=2}^{m-1} \sum_{t=1}^{b_{m,k}} d_{m,k,t} \sum_{\substack{1 \leq j_1, j_2, \dots, j_k \leq n \\ (\text{distinct})}} f_{m,k,t}(A, j_1, j_2, \dots, j_k). \end{aligned} \quad (31)$$

Plugging (31) into (25), we obtain:

$$C_m = \text{tr}(A^m) - \sum_{k=2}^{m-1} \sum_{t=1}^{b_{m,k}} d_{m,k,t} \sum_{\substack{1 \leq j_1, j_2, \dots, j_k \leq n \\ (\text{distinct})}} f_{m,k,t}(A, j_1, j_2, \dots, j_k), \quad (32)$$

which completes the proof. $\square$

A.4 Proof of Lemma 3

It is sufficient to show that the value of the Full-Sum remains the same before and after either types of the pruning.

Consider the Type I pruning first. Without loss of generality, suppose node 1 is the Pendant and node 2 is the Hinge. Let v and u be their respective node labels, and $M^{(1)}, M^{(2)}, \dots, M^{(s)}$ be the labels of the edges between them (after orientation alignment). We can express the Full-Sum of the original FAM $\mathcal{G}$ as:

$$\begin{aligned} \text{FS}(\mathcal{G}) &= \sum_{j_1, j_2, \dots, j_k} f^{(1,2)}(A, j_3, \dots, j_k) \cdot u_{j_2} \cdot \prod_{l=1}^s M_{j_2 j_1}^{(l)} \cdot v_{j_1} \\ &= \sum_{j_2, \dots, j_k} f^{(1,2)}(A, j_3, \dots, j_k) \cdot u_{j_2} \cdot \sum_{j_1} \left(\prod_{l=1}^s M_{j_2 j_1}^{(l)} \right) \cdot v_{j_1} \end{aligned}$$

where $f^{(1,2)}(A, j_3, \dots, j_k)$ captures all other terms in the product that are independent of j_1 and j_2 .

Let $M = M^{(1)} \circ M^{(2)} \circ \dots \circ M^{(s)}$ denote the Hadamard product of edge matrices. Then:

$$\begin{aligned} \text{FS}(\mathcal{G}) &= \sum_{j_2, \dots, j_k} f^{(1,2)}(A, j_3, \dots, j_k) \cdot u_{j_2} \cdot \sum_{j_1} M_{j_2 j_1} \cdot v_{j_1} \\ &= \sum_{j_2, \dots, j_k} f^{(1,2)}(A, j_3, \dots, j_k) \cdot [u \circ (M \cdot v)]_{j_2} \end{aligned} \quad (33)$$

We can see that after pruning, the new FAM $\mathcal{G}^{\text{new}}$ has node 1 removed and the hinge node 2 now has an updated label $u^{\text{new}} = u \circ (M \cdot v)$. Its Full-Sum is:

$$\begin{aligned} \text{FS}(\mathcal{G}^{\text{new}}) &= \sum_{j_2, \dots, j_k} f^{(1,2)}(A, j_3, \dots, j_k) \cdot u_{j_2}^{\text{new}} \\ &= \sum_{j_2, \dots, j_k} f^{(1,2)}(A, j_3, \dots, j_k) \cdot [u \circ (M \cdot v)]_{j_2} \end{aligned} \quad (34)$$

Therefore, $\text{FS}(\mathcal{G}) = \text{FS}(\mathcal{G}^{\text{new}})$.

Consider the Type II pruning. Without loss of generality, suppose node 1 is the Pendant, and nodes 2 and 3 are its Hinges. Let y , x , and z be their respective node labels. Let $Q^{(1)}, \dots, Q^{(s)}$ be the labels of edges between nodes 1 and 2, and $R^{(1)}, \dots, R^{(t)}$ be the labels of edges between nodes 1 and 3, both after orientation alignment.

We can express the Full-Sum of the original FAM $\mathcal{G}$ as:

$$\text{FS}(\mathcal{G}) = \sum_{j_1, \dots, j_k} f^{(1,2,3)}(A, j_4, \dots, j_k) \cdot x_{j_2} \cdot \left(\prod_{l=1}^s Q_{j_1 j_2}^{(l)} \right) \cdot y_{j_1} \cdot \left(\prod_{l=1}^t R_{j_1 j_3}^{(l)} \right) \cdot z_{j_3}$$

where $f^{(1,2,3)}(A, j_4, \dots, j_k)$ captures all other terms independent of j_1 , j_2 , and j_3 . Let $Q = Q^{(1)} \circ \dots \circ Q^{(s)}$ and $R = R^{(1)} \circ \dots \circ R^{(t)}$. Then:

$$\begin{aligned} \text{FS}(\mathcal{G}) &= \sum_{j_2, j_3, \dots, j_k} f^{(1,2,3)}(A, j_4, \dots, j_k) \cdot x_{j_2} \cdot z_{j_3} \cdot \sum_{j_1} Q_{j_1 j_2} \cdot y_{j_1} \cdot R_{j_1 j_3} \\ &= \sum_{j_2, j_3, \dots, j_k} f^{(1,2,3)}(A, j_4, \dots, j_k) \cdot x_{j_2} \cdot z_{j_3} \cdot [Q^\top \cdot d(y) \cdot R]_{j_2 j_3} \end{aligned} \quad (35)$$

where $d(y)$ is the diagonal matrix with entries y_{j_1} on the diagonal.

After pruning, the new FAM $\mathcal{G}^{\text{new}}$ has node 1 removed and a new edge with label $E = Q^\top \cdot d(y) \cdot R$ between nodes 2 and 3. Its Full-Sum is:

$$\begin{aligned} \text{FS}(\mathcal{G}^{\text{new}}) &= \sum_{j_2, j_3, \dots, j_k} f^{(1,2,3)}(A, j_4, \dots, j_k) \cdot x_{j_2} \cdot E_{j_2 j_3} \cdot z_{j_3} \\ &= \sum_{j_2, j_3, \dots, j_k} f^{(1,2,3)}(A, j_4, \dots, j_k) \cdot x_{j_2} \cdot [Q^\top \cdot d(y) \cdot R]_{j_2 j_3} \cdot z_{j_3} \end{aligned} \quad (36)$$

Therefore, we also have $\text{FS}(\mathcal{G}) = \text{FS}(\mathcal{G}^{\text{new}})$.

In both cases, the pruning operation reduces the number of nodes by exactly 1 while preserving the value of the Full-Sum. This completes the proof. $\square$

A.5 Proof of Theorem 4

Before we prove Theorem 4, we show that the pruning algorithm eventually stops.

Consider each pruning operation. Since we remove exactly 1 node in each pruning operation, the number of nodes reduces by exactly 1 before and after each pruning operation. Consider the termination condition. The algorithm terminates either when only one node remains or no prunable nodes (pendants) remain. In either case, when the algorithm terminates, there is at least one node remaining. Therefore, since the algorithm starts with a finite number of nodes k , we conclude that the algorithm must terminate after at most $(k - 1)$ pruning steps, which shows that the pruning algorithm eventually stops.

We now proceed to prove Theorem 4. Consider the first case. Let v be the node label for the only one node remaining in the final FAM. There is no other node for the only remaining node to connect to, therefore there is no edge in the final FAM. Hence, the Full-Sum simplifies to:

$$\text{FS}(\mathcal{G}) = \sum_{j_1} v_{j_1} = \mathbf{1}'_n v \quad (37)$$

This is a SEA term by definition, since it is expressible in a succinct algebraic form without any remaining sums over indices.

Consider the second case. In this case, no pendant node of either type is left. First we state a useful lemma.

Lemma 9. *The pruning algorithm preserves the connectivity of the FAM. That is, if a FAM is connected, then after one step of either Type I pruning or Type II pruning, the pruned FAM is still connected.*

Proof of Lemma 9: Consider Type I pruning. Type I pruning deletes a node with 1 immediate neighbor, which does not affect the connectivity of the remaining nodes. This shows that after one step of Type I pruning, the pruned FAM is still connected.

Consider Type II pruning. Type II pruning deletes a node with 2 immediate neighbors, which does not affect the connectivity of the remaining nodes. This shows that after one step of Type II pruning, the pruned FAM is still connected. $\square$

We now proceed to prove Theorem 4. We first show that each of the remaining node in the final FAM has at least 3 immediate neighbors. We prove this by contradiction. Assume that there exists a node with fewer than 3 immediate neighbors. If it has 2 neighbors, it would be a Type II pendant, contradicting our assumption that no pendant nodes remain. If it has 1 neighbor, it would be a Type I pendant, contradicting our assumption that no pendant nodes remain. If it has 0 neighbor, then the FAM would be disconnected. By Lemma 6, the original multi-graph is connected. By Lemma 9,

the pruning process preserves connectivity of the FAM. Therefore, when the algorithm terminates, the final FAM must be connected. This leads to a contradiction. Therefore, we conclude that each of the remaining node in the final FAM has at least 3 immediate neighbors.

We then show that the final FAM has at least 4 nodes. Pick a node in the final FAM. Using the above argument, it has at least 3 immediate neighbors. Consider the picked node and three of its immediate neighbors in the final FAM. These are 4 different nodes in the final FAM, therefore we conclude that there must be at least 4 nodes remaining in the final FAM.

Finally, we show that the FS term has at most $\lfloor m/2 \rfloor$ layers of sum. We call a node in the original FAM prunable, if it does not remain in the final FAM, i.e. it is pruned during a step of pruning operation. Denote k the number of nodes in the original FAM, denote p the number of prunable nodes in the original FAM, and denote d_2 the number of degree 2 nodes in the original FAM.

Consider the number of prunable nodes in the original FAM. Note that every node with degree 2 must be prunable, since they can have no more than 2 neighbors. Therefore, we conclude that

$$p \geq d_2. \quad (38)$$

Consider the sum of degrees of all nodes in the original FAM. Since the original FAM has m edges, the sum of degrees of all nodes in the original FAM is $2m$. By Lemma 6, the original FAM is connected, and every node in the original FAM has even degrees. This means that the degree of each node in the original FAM can only be positive even numbers. Therefore, any node whose degree is not 2 has a degree of at least 4. Therefore, the sum of degrees of all nodes in the original FAM is no less than $2d_2 + 4(k - d_2)$. Hence, we have the following inequality:

$$2m \geq 2d_2 + 4(k - d_2). \quad (39)$$

By (39), we conclude that $4(k - d_2) \leq 2m$. Therefore, by (38), we have

$$k - p \leq k - d_2 \leq m/2. \quad (40)$$

Since the number of remaining nodes in the final FAM must be an integer, we conclude that the number of remaining nodes in the final FAM is no greater than $\lfloor m/2 \rfloor$. Since each node in the final FAM corresponds to one layer of summation, we conclude that the FS term has at most $\lfloor m/2 \rfloor$ layers of sum. This completes the proof. $\square$

B Details of The LLMs under investigation

While we primarily use DeepSeek-R1 in our hugAI approach, we have also tested 11 other models. Their relevant information is summarized in Table 6.

Table 6: The 11 LLMs under comparison. All models are queried in their “reasoning” or “thinking” mode where available.

Model	Provider	Reasoning mode	API identifier
Claude Opus 4.8	Anthropic	Extended thinking	anthropic/claude-opus-4.8
Claude Sonnet 4.6	Anthropic	Extended thinking	anthropic/claude-sonnet-4.6
DeepSeek V4 Pro	DeepSeek	High effort	deepseek/deepseek-v4-pro
GPT-5.5	OpenAI	High effort	openai/gpt-5.5
Gemini 3.1 Pro	Google	Thinking mode	google/gemini-3.1-pro-preview
Gemini 3.5 Flash	Google	Thinking mode	google/gemini-3.5-flash
GLM 5.1	Zhipu AI	Default	z-ai/glm-5.1
Kimi K2.6	Moonshot	Default	moonshotai/kimi-k2.6
MiniMax-M3	MiniMax	Default	minimax/minimax-m3
MiniMax-M2.7	MiniMax	Default	minimax/minimax-m2.7
DeepSeek V3.2	DeepSeek	High effort	deepseek/deepseek-v3.2

C Prompts

The complete two-stage prompt used in the evaluation is reproduced below; see Figures 11-12. Stage 1 is sent first; after structural verification, Stage 2 is sent with the model’s own Stage 1 code included as context.

System Prompt (Shared by Both Stages)

You are a mathematician implementing a combinatorial algorithm. Your task is to derive an explicit formula for a cycle count statistic and output it as executable Python code. Follow the algorithm described below precisely, and produce code output as instructed.

Stage 1 User Prompt: Merging Process

PROBLEM. For a symmetric matrix A in $\mathbb{R}^{n \times n}$ with zero diagonal ($A_{ii} = 0$), the order- m cycle count statistic is:

$$C_m(A) = \sum_{i_1, \dots, i_m \text{ all distinct}} A_{i_1, i_2} * A_{i_2, i_3} * \dots * A_{i_m, i_1}$$

The brute-force cost is $O(n^m)$. We want a Computationally Efficient Equivalent Form (CEEF): a formula expressing C_m as a linear combination of terms, each computable in polynomial time using matrix operations.

YOUR TASK (Step 1 of 2): Implement the MERGING PROCESS that decomposes C_m into canonical multi-graph types with aggregated Mobius coefficients. Your code must accept m as a parameter and work for any $m \geq 2$.

STEP 1: MERGING PROCESS (Partitions $\rightarrow$ Multi-graphs $\rightarrow$ Canonical Types)

1(a) Generate all multi-graphs.

- Fix the index set $S = \{i_1, \dots, i_m\}$ with a cycle graph: edges $i_1-i_2, i_2-i_3, \dots, i_m-i_1$.
- A partition σ of S merges some indices, producing a multi-graph on k nodes. The edge weight $w(a,b)$ equals the total number of edges (in both directions) between blocks a and b after merging.
- Exclude any multi-graph with self-loops ($A_{ii} = 0$).

1(b) Check graph isomorphism.

- Group multi-graphs by simultaneous row-and-column permutation of the edge weight matrix. Pick the lexicographically smallest representative.

1(c) Find linear coefficients.

- $\mu(\text{Ohat}, \sigma) = (-1)^{m-k} * \prod_{i=1}^k (|B_i| - 1)!$
- Sum these Mobius values over all partitions in the same isomorphism class.

>>> CODE OUTPUT <<<

Write a self-contained Python script defining `step1(m)` that returns a list of (`weight_matrix`, `coefficient`) pairs.

Sanity checks:

- $m=2$: 1 surviving partition, 1 canonical type
- $m=3$: 1 surviving partition, 1 canonical type
- $m=4$: 4 surviving partitions, 3 canonical types

REMINDERS:

- In Stage 1, graph isomorphism is checked using the symmetric edge-weight matrix, because before pruning all edge labels are inherited from the symmetric matrix A .
- The Stage 1 `weight_matrix` is only a canonical pre-pruning representation.
- Stage 2 will convert each `weight_matrix` into FAM edge records before pruning.
- The cycle wraps around: position m connects back to position 1.
- Self-loop filtering is essential.

Figure 11: System prompt and Stage 1 (Merging Process) of the benchmark evaluation prompt.

D The output CEEF formulas

We now present the complete formulas of C_m that we have calculated for $m = 3, 4, \dots, 12$ as follows. We have grouped the terms according to the node count of the original multi-graph.

D.1 $m=3$

$C_3 =$
Unique graphs for $k=3$: (1 graphs)
 $+1 \quad \mathbf{1}'_n \cdot ((A \circ (A \cdot A)) \cdot \mathbf{1}_n)$

D.2 m=4

$C_4 =$

Unique graphs for k=2: (1 graphs)

$$+1 \quad \mathbf{1}'_n \cdot ((A \circ A \circ A \circ A) \cdot \mathbf{1}_n)$$

Unique graphs for k=3: (1 graphs)

$$-2 \quad \mathbf{1}'_n \cdot (((A \circ A) \cdot \mathbf{1}_n) \circ ((A \circ A) \cdot \mathbf{1}_n))$$

Unique graphs for k=4: (1 graphs)

$$+1 \quad \mathbf{1}'_n \cdot ((A \circ ((A \cdot A) \cdot A)) \cdot \mathbf{1}_n)$$

D.3 m=5

$C_5 =$

Unique graphs for k=3: (1 graphs)

$$+5 \quad \mathbf{1}'_n \cdot ((A \circ A \circ A \circ (A \cdot A)) \cdot \mathbf{1}_n)$$

Unique graphs for k=4: (1 graphs)

$$-5 \quad \mathbf{1}'_n \cdot (((A \circ A) \cdot \mathbf{1}_n) \circ ((A \circ (A \cdot A)) \cdot \mathbf{1}_n))$$

Unique graphs for k=5: (1 graphs)

$$+1 \quad \mathbf{1}'_n \cdot ((A \circ (((A \cdot A) \cdot A) \cdot A)) \cdot \mathbf{1}_n)$$

D.4 m=6

$C_6 =$

Unique graphs for k=2: (1 graphs)

$$+4 \quad \mathbf{1}'_n \cdot ((A \circ A \circ A \circ A \circ A \circ A) \cdot \mathbf{1}_n)$$

Unique graphs for k=3: (2 graphs)

$$-12 \quad \mathbf{1}'_n \cdot (((A \circ A) \cdot \mathbf{1}_n) \circ ((A \circ A \circ A \circ A) \cdot \mathbf{1}_n))$$

$$-4 \quad \mathbf{1}'_n \cdot ((A \circ A \circ ((A \circ A) \cdot (A \circ A))) \cdot \mathbf{1}_n)$$

Unique graphs for k=4: (4 graphs)

$$+6 \quad \mathbf{1}'_n \cdot ((A \circ A \circ A \circ ((A \cdot A) \cdot A)) \cdot \mathbf{1}_n)$$

$$+4 \quad \mathbf{1}'_n \cdot (((A \circ A) \cdot \mathbf{1}_n) \circ ((A \circ A) \cdot \mathbf{1}_n)) \circ ((A \circ A) \cdot \mathbf{1}_n)$$

$$+3 \quad \mathbf{1}'_n \cdot (((A \circ A) \cdot ((A \circ A) \cdot \mathbf{1}_n)) \circ ((A \circ A) \cdot \mathbf{1}_n))$$

$$+9 \quad \mathbf{1}'_n \cdot ((A \circ A \circ (A \cdot A) \circ (A \cdot A)) \cdot \mathbf{1}_n)$$

Unique graphs for k=5: (2 graphs)

$$-6 \quad \mathbf{1}'_n \cdot (((A \circ A) \cdot \mathbf{1}_n) \circ ((A \circ ((A \cdot A) \cdot A)) \cdot \mathbf{1}_n))$$

$$-3 \quad \mathbf{1}'_n \cdot (((A \circ (A \cdot A)) \cdot \mathbf{1}_n) \circ ((A \circ (A \cdot A)) \cdot \mathbf{1}_n))$$

Unique graphs for k=6: (1 graphs)

$$+1 \quad \mathbf{1}'_n \cdot ((A \circ (((A \cdot A) \cdot A) \cdot A) \cdot A)) \cdot \mathbf{1}_n)$$

D.5 m=7

$C_7 =$

Unique graphs for k=3: (2 graphs)

$$+28 \quad \mathbf{1}'_n \cdot ((A \circ A \circ A \circ A \circ A \circ (A \cdot A)) \cdot \mathbf{1}_n)$$

$$+28 \quad \mathbf{1}'_n \cdot ((A \circ A \circ A \circ ((A \circ A \circ A) \cdot A)) \cdot \mathbf{1}_n)$$

Unique graphs for k=4: (4 graphs)

$$-14 \quad \mathbf{1}'_n \cdot (((A \circ A \circ A \circ A) \cdot \mathbf{1}_n) \circ ((A \circ (A \cdot A)) \cdot \mathbf{1}_n))$$

$$-56 \quad \mathbf{1}'_n \cdot (((A \circ A) \cdot \mathbf{1}_n) \circ ((A \circ A \circ A \circ (A \cdot A)) \cdot \mathbf{1}_n))$$

$$-7 \quad \mathbf{1}'_n \cdot ((A \circ A \circ A \circ (A \cdot d((A \circ A) \cdot \mathbf{1}_n) \cdot A)) \cdot \mathbf{1}_n)$$

$$-28 \quad \mathbf{1}'_n \cdot ((A \circ A \circ ((A \circ (A \cdot A)) \cdot (A \circ A))) \cdot \mathbf{1}_n)$$

Unique graphs for k=5: (6 graphs)

$$+7 \quad \mathbf{1}'_n \cdot ((A \circ A \circ A \circ ((A \cdot A) \cdot A)) \cdot \mathbf{1}_n)$$

$$+14 \quad \mathbf{1}'_n \cdot (((A \circ A) \cdot \mathbf{1}_n) \circ ((A \circ A) \cdot \mathbf{1}_n)) \circ ((A \circ (A \cdot A)) \cdot \mathbf{1}_n)$$

$$+7 \quad \mathbf{1}'_n \cdot (((A \circ A) \cdot ((A \circ A) \cdot \mathbf{1}_n)) \circ ((A \circ (A \cdot A)) \cdot \mathbf{1}_n))$$

$$+21 \quad \mathbf{1}'_n \cdot ((A \circ A \circ ((A \cdot A) \cdot A) \circ (A \cdot A)) \cdot \mathbf{1}_n)$$

$$+7 \quad \mathbf{1}'_n \cdot (((A \circ A) \cdot \mathbf{1}_n) \circ ((A \circ (A \cdot A)) \cdot ((A \circ A) \cdot \mathbf{1}_n)))$$

$$+7 \quad \mathbf{1}'_n \cdot ((A \circ (A \cdot A) \circ (A \cdot A) \circ (A \cdot A)) \cdot \mathbf{1}_n)$$

Unique graphs for k=6: (2 graphs)

$$-7 \quad \mathbf{1}'_n \cdot (((A \circ A) \cdot \mathbf{1}_n) \circ ((A \circ ((A \cdot A) \cdot A) \cdot A)) \cdot \mathbf{1}_n))$$

$$-7 \quad \mathbf{1}'_n \cdot (((A \circ ((A \cdot A) \cdot A)) \cdot \mathbf{1}_n) \circ ((A \circ (A \cdot A)) \cdot \mathbf{1}_n))$$

Unique graphs for k=7: (1 graphs)

$$+1 \quad \mathbf{1}'_n \cdot ((A \circ (((A \cdot A) \cdot A) \cdot A) \cdot A)) \cdot \mathbf{1}_n)$$

D.6 m=8

$C_8 =$

Unique graphs for k=2: (1 graphs)

$$+36 \quad 1'_n \cdot ((A \circ A \circ A \circ A \circ A \circ A \circ A) \cdot 1_n)$$

Unique graphs for k=3: (3 graphs)

$$-96 \quad 1'_n \cdot (((A \circ A) \cdot 1_n) \circ ((A \circ A \circ A \circ A \circ A) \cdot 1_n))$$

$$-36 \quad 1'_n \cdot (((A \circ A \circ A \circ A) \cdot 1_n) \circ ((A \circ A \circ A \circ A) \cdot 1_n))$$

$$-112 \quad 1'_n \cdot ((A \circ A \circ A \circ A \circ ((A \circ A) \cdot (A \circ A))) \cdot 1_n)$$

Unique graphs for k=4: (11 graphs)

$$+32 \quad 1'_n \cdot ((A \circ A \circ A \circ A \circ A \circ ((A \cdot A) \cdot A)) \cdot 1_n)$$

$$+72 \quad 1'_n \cdot (((((A \circ A) \cdot 1_n) \circ ((A \circ A) \cdot 1_n)) \circ ((A \circ A \circ A \circ A) \cdot 1_n)))$$

$$+16 \quad 1'_n \cdot (((A \circ A) \cdot ((A \circ A) \cdot 1_n)) \circ ((A \circ A \circ A \circ A) \cdot 1_n))$$

$$+80 \quad 1'_n \cdot ((A \circ A \circ A \circ A \circ A \circ (A \cdot A) \circ (A \cdot A)) \cdot 1_n)$$

$$+32 \quad 1'_n \cdot (((A \circ A) \cdot 1_n) \circ ((A \circ A \circ A \circ A) \cdot ((A \circ A) \cdot 1_n)))$$

$$+192 \quad 1'_n \cdot ((A \circ A \circ A \circ ((A \circ A \circ (A \cdot A)) \cdot A)) \cdot 1_n)$$

$$+32 \quad 1'_n \cdot ((A \circ A \circ A \circ ((A \cdot A) \cdot (A \circ A \circ A))) \cdot 1_n)$$

$$+4 \quad 1'_n \cdot ((A \circ A \circ A \circ ((A \cdot (A \circ A \circ A)) \cdot A)) \cdot 1_n)$$

$$+64 \quad 1'_n \cdot (((A \circ A) \cdot 1_n) \circ ((A \circ A \circ ((A \circ A) \cdot (A \circ A))) \cdot 1_n))$$

$$+5 \quad 1'_n \cdot ((A \circ A \circ ((A \circ A) \cdot ((A \circ A) \cdot (A \circ A)))) \cdot 1_n)$$

$$+22 \quad \sum_{i_1 i_2 i_3 i_4} (A \circ A)_{i_1 i_2} A_{i_1 i_3} A_{i_1 i_4} A_{i_2 i_3} A_{i_2 i_4} (A \circ A)_{i_3 i_4}$$

Unique graphs for k=5: (13 graphs)

$$-16 \quad 1'_n \cdot (((A \circ A \circ A \circ A) \cdot 1_n) \circ ((A \circ ((A \cdot A) \cdot A)) \cdot 1_n))$$

$$-64 \quad 1'_n \cdot (((A \circ (A \cdot A)) \cdot 1_n) \circ ((A \circ A \circ A \circ (A \cdot A)) \cdot 1_n))$$

$$-8 \quad 1'_n \cdot ((A \circ A \circ A \circ (A \cdot d((A \circ (A \cdot A)) \cdot 1_n) \cdot A)) \cdot 1_n)$$

$$-64 \quad 1'_n \cdot ((A \circ A \circ A \circ ((A \cdot A) \cdot A)) \cdot ((A \circ A) \cdot 1_n))$$

$$-16 \quad 1'_n \cdot ((A \circ A \circ A \circ ((A \cdot A) \cdot d((A \circ A) \cdot 1_n) \cdot A)) \cdot 1_n)$$

$$-12 \quad 1'_n \cdot (((((A \circ A) \cdot 1_n) \circ ((A \circ A) \cdot 1_n)) \circ ((A \circ A) \cdot 1_n)) \circ ((A \circ A) \cdot 1_n))$$

$$-16 \quad 1'_n \cdot (((((A \circ A) \cdot ((A \circ A) \cdot 1_n)) \circ ((A \circ A) \cdot 1_n)) \circ ((A \circ A) \cdot 1_n)))$$

$$-96 \quad 1'_n \cdot (((A \circ A) \cdot 1_n) \circ ((A \circ A \circ (A \cdot A) \circ (A \cdot A)) \cdot 1_n))$$

$$-4 \quad 1'_n \cdot (((A \circ A) \cdot ((A \circ A) \cdot ((A \circ A) \cdot 1_n))) \circ ((A \circ A) \cdot 1_n))$$

$$-24 \quad 1'_n \cdot ((A \circ A \circ (A \cdot d((A \circ A) \cdot 1_n) \cdot A) \circ (A \cdot A)) \cdot 1_n)$$

$$-32 \quad 1'_n \cdot ((A \circ A \circ ((A \circ ((A \cdot A) \cdot A)) \cdot (A \circ A))) \cdot 1_n)$$

$$-64 \quad 1'_n \cdot ((A \circ A \circ ((A \circ (A \cdot A)) \cdot (A \circ (A \cdot A)))) \cdot 1_n)$$

$$-16 \quad 1'_n \cdot ((A \circ A \circ (((A \cdot A) \circ (A \cdot A)) \cdot (A \circ A))) \cdot 1_n)$$

Unique graphs for k=6: (12 graphs)

$$+8 \quad 1'_n \cdot ((A \circ A \circ A \circ (((A \cdot A) \cdot A) \cdot A)) \cdot 1_n)$$

$$+16 \quad 1'_n \cdot (((((A \circ A) \cdot 1_n) \circ ((A \circ A) \cdot 1_n)) \circ ((A \circ ((A \cdot A) \cdot A)) \cdot 1_n)))$$

$$+8 \quad 1'_n \cdot (((A \circ A) \cdot ((A \circ A) \cdot 1_n)) \circ ((A \circ ((A \cdot A) \cdot A)) \cdot 1_n))$$

$$+16 \quad 1'_n \cdot (((((A \circ A) \cdot 1_n) \circ ((A \circ (A \cdot A)) \cdot 1_n)) \circ ((A \circ (A \cdot A)) \cdot 1_n)))$$

$$+24 \quad 1'_n \cdot ((A \circ A \circ (((A \cdot A) \cdot A) \cdot A) \circ (A \cdot A)) \cdot 1_n)$$

$$+12 \quad 1'_n \cdot ((A \circ A \circ ((A \cdot A) \cdot A) \circ (A \cdot (A \cdot A))) \cdot 1_n)$$

$$+8 \quad 1'_n \cdot (((A \circ A) \cdot 1_n) \circ ((A \circ ((A \cdot A) \cdot A)) \cdot ((A \circ A) \cdot 1_n)))$$

$$+16 \quad 1'_n \cdot (((A \circ (A \cdot A)) \cdot ((A \circ A) \cdot 1_n)) \circ ((A \circ (A \cdot A)) \cdot 1_n))$$

$$+4 \quad 1'_n \cdot (((A \circ A) \cdot 1_n) \circ ((A \circ ((A \cdot A) \cdot d((A \circ A) \cdot 1_n) \cdot A)) \cdot 1_n))$$

$$+4 \quad 1'_n \cdot (((A \circ A) \cdot ((A \circ (A \cdot A)) \cdot 1_n)) \circ ((A \circ (A \cdot A)) \cdot 1_n))$$

$$+24 \quad 1'_n \cdot ((A \circ ((A \cdot A) \cdot A) \circ (A \cdot A) \circ (A \cdot A)) \cdot 1_n)$$

$$+2 \quad 1'_n \cdot ((A \circ (((A \cdot A) \circ (A \cdot A) \circ (A \cdot A)) \cdot A)) \cdot 1_n)$$

Unique graphs for k=7: (3 graphs)

$$-8 \quad 1'_n \cdot (((A \circ A) \cdot 1_n) \circ ((A \circ (((A \cdot A) \cdot A) \cdot A)) \cdot 1_n))$$

$$-8 \quad 1'_n \cdot (((A \circ (((A \cdot A) \cdot A) \cdot A)) \cdot 1_n) \circ ((A \circ (A \cdot A)) \cdot 1_n))$$

$$-4 \quad 1'_n \cdot (((A \circ ((A \cdot A) \cdot A)) \cdot 1_n) \circ ((A \circ ((A \cdot A) \cdot A)) \cdot 1_n))$$

Unique graphs for k=8: (1 graphs)

$$+1 \quad 1'_n \cdot ((A \circ ((((((A \cdot A) \cdot A) \cdot A) \cdot A) \cdot A) \cdot A)) \cdot 1_n)$$

D.7 m=9

$C_9 =$

Unique graphs for k=3: (3 graphs)

$$+324 \quad 1'_n \cdot ((A \circ A \circ A \circ A \circ A \circ A \circ A \circ (A \cdot A)) \cdot 1_n)$$

$$+648 \quad 1'_n \cdot ((A \circ A \circ A \circ A \circ A \circ ((A \circ A \circ A) \cdot A)) \cdot 1_n)$$

$$+176 \quad 1'_n \cdot ((A \circ A \circ A \circ ((A \circ A \circ A) \cdot (A \circ A \circ A))) \cdot 1_n)$$

$$\begin{aligned}
-144 & \quad 1'_n \cdot ((A \circ A \circ ((A \circ ((A \cdot A) \cdot A)) \cdot (A \circ (A \cdot A)))) \cdot 1_n) \\
-54 & \quad 1'_n \cdot ((A \circ A \circ ((A \cdot A) \cdot d((A \circ A) \cdot 1_n) \cdot A) \circ (A \cdot A)) \cdot 1_n) \\
-36 & \quad 1'_n \cdot ((A \circ A \circ (((A \cdot A) \cdot A) \circ (A \cdot A)) \cdot (A \circ A))) \cdot 1_n) \\
-72 & \quad 1'_n \cdot ((A \circ A \circ ((A \circ (A \cdot A)) \cdot ((A \cdot A) \circ (A \cdot A)))) \cdot 1_n) \\
-18 & \quad 1'_n \cdot (((A \circ A) \cdot 1_n) \circ ((A \circ (A \cdot A)) \cdot ((A \circ A) \cdot ((A \circ A) \cdot 1_n)))) \\
-48 & \quad 1'_n \cdot ((A \circ (A \cdot A) \circ ((A \circ (A \cdot A)) \cdot (A \circ (A \cdot A)))) \cdot 1_n) \\
\text{Unique graphs for } k=7: & \text{ (17 graphs)} \\
+9 & \quad 1'_n \cdot ((A \circ A \circ A \circ (((((A \cdot A) \cdot A) \cdot A) \cdot A) \cdot A)) \cdot 1_n) \\
+18 & \quad 1'_n \cdot (((((A \circ A) \cdot 1_n) \circ ((A \circ A) \cdot 1_n)) \circ ((A \circ (((A \cdot A) \cdot A) \cdot A)) \cdot 1_n)) \\
+9 & \quad 1'_n \cdot (((A \circ A) \cdot ((A \circ A) \cdot 1_n)) \circ ((A \circ (((A \cdot A) \cdot A) \cdot A)) \cdot 1_n) \\
+36 & \quad 1'_n \cdot (((((A \circ A) \cdot 1_n) \circ ((A \circ ((A \cdot A) \cdot A)) \cdot 1_n)) \circ ((A \circ (A \cdot A)) \cdot 1_n)) \\
+27 & \quad 1'_n \cdot ((A \circ A \circ (((((A \cdot A) \cdot A) \cdot A) \cdot A) \circ (A \cdot A)) \cdot 1_n) \\
+27 & \quad 1'_n \cdot ((A \circ A \circ (((A \cdot A) \cdot A) \cdot A) \circ (A \cdot (A \cdot A))) \cdot 1_n) \\
+9 & \quad 1'_n \cdot (((A \circ A) \cdot 1_n) \circ ((A \circ (((A \cdot A) \cdot A) \cdot A)) \cdot ((A \circ A) \cdot 1_n))) \\
+18 & \quad 1'_n \cdot (((A \circ ((A \cdot A) \cdot A)) \cdot ((A \circ A) \cdot 1_n)) \circ ((A \circ (A \cdot A)) \cdot 1_n)) \\
+18 & \quad 1'_n \cdot (((A \circ (A \cdot A)) \cdot ((A \circ A) \cdot 1_n)) \circ ((A \circ ((A \cdot A) \cdot A)) \cdot 1_n)) \\
+9 & \quad 1'_n \cdot (((A \circ A) \cdot 1_n) \circ ((A \circ (((A \cdot A) \cdot A) \cdot d((A \circ A) \cdot 1_n) \cdot A)) \cdot 1_n)) \\
+9 & \quad 1'_n \cdot (((A \circ A) \cdot 1_n) \circ ((A \circ ((A \cdot A) \cdot d((A \circ (A \cdot A)) \cdot 1_n) \cdot A)) \cdot 1_n)) \\
+9 & \quad 1'_n \cdot (((A \circ A) \cdot ((A \circ (A \cdot A) \cdot A)) \cdot 1_n)) \circ ((A \circ (A \cdot A)) \cdot 1_n) \\
+6 & \quad 1'_n \cdot (((((A \circ (A \cdot A)) \cdot 1_n) \circ ((A \circ (A \cdot A)) \cdot 1_n)) \circ ((A \circ (A \cdot A)) \cdot 1_n)) \\
+27 & \quad 1'_n \cdot ((A \circ (((A \cdot A) \cdot A) \cdot A) \circ (A \cdot A) \circ (A \cdot A)) \cdot 1_n) \\
+27 & \quad 1'_n \cdot ((A \circ ((A \cdot A) \cdot A) \circ (A \cdot (A \cdot A)) \circ (A \cdot A)) \cdot 1_n) \\
+9 & \quad 1'_n \cdot (((A \circ (A \cdot A)) \cdot ((A \circ (A \cdot A)) \cdot 1_n)) \circ ((A \circ (A \cdot A)) \cdot 1_n)) \\
+9 & \quad 1'_n \cdot ((A \circ (((A \cdot A) \cdot A) \circ (A \cdot A) \circ (A \cdot A)) \cdot A)) \cdot 1_n) \\
\text{Unique graphs for } k=8: & \text{ (3 graphs)} \\
-9 & \quad 1'_n \cdot (((A \circ A) \cdot 1_n) \circ ((A \circ (((((A \cdot A) \cdot A) \cdot A) \cdot A) \cdot A)) \cdot 1_n)) \\
-9 & \quad 1'_n \cdot (((A \circ (((A \cdot A) \cdot A) \cdot A) \cdot A)) \cdot 1_n) \circ ((A \circ (A \cdot A)) \cdot 1_n)) \\
-9 & \quad 1'_n \cdot (((A \circ (((A \cdot A) \cdot A) \cdot A)) \cdot 1_n) \circ ((A \circ ((A \cdot A) \cdot A)) \cdot 1_n)) \\
\text{Unique graphs for } k=9: & \text{ (1 graphs)} \\
+1 & \quad 1'_n \cdot ((A \circ (((((((A \cdot A) \cdot A) \cdot A) \cdot A) \cdot A) \cdot A) \cdot A)) \cdot 1_n)
\end{aligned}$$

D.8 m=10

$C_{10} =$

Unique graphs for k=2: (1 graphs)

$$+576 \quad 1'_n \cdot ((A \circ A \circ A \circ A \circ A \circ A \circ A \circ A \circ A \circ A) \cdot 1_n)$$

Unique graphs for k=3: (4 graphs)

$$-1440 \quad 1'_n \cdot (((A \circ A) \cdot 1_n) \circ ((A \circ A \circ A \circ A \circ A \circ A \circ A \circ A) \cdot 1_n))$$

$$-960 \quad 1'_n \cdot (((A \circ A \circ A \circ A) \cdot 1_n) \circ ((A \circ A \circ A \circ A \circ A \circ A \circ A) \cdot 1_n))$$

$$-1800 \quad 1'_n \cdot ((A \circ A \circ A \circ A \circ A \circ A \circ A \circ ((A \circ A) \cdot (A \circ A))) \cdot 1_n)$$

$$-2160 \quad 1'_n \cdot ((A \circ A \circ A \circ A \circ A \circ ((A \circ A \circ A \circ A) \cdot (A \circ A))) \cdot 1_n)$$

Unique graphs for k=4: (23 graphs)

$$+360 \quad 1'_n \cdot ((A \circ A \circ A \circ A \circ A \circ A \circ A \circ A \circ ((A \cdot A) \cdot A)) \cdot 1_n)$$

$$+960 \quad 1'_n \cdot (((((A \circ A) \cdot 1_n) \circ ((A \circ A) \cdot 1_n)) \circ ((A \circ A \circ A \circ A \circ A \circ A) \cdot 1_n))$$

$$+120 \quad 1'_n \cdot (((A \circ A) \cdot ((A \circ A) \cdot 1_n)) \circ ((A \circ A \circ A \circ A \circ A \circ A) \cdot 1_n))$$

$$+1260 \quad 1'_n \cdot ((A \circ A \circ A \circ A \circ A \circ A \circ (A \cdot A) \circ (A \cdot A)) \cdot 1_n)$$

$$+540 \quad 1'_n \cdot (((A \circ A) \cdot 1_n) \circ ((A \circ A \circ A \circ A \circ A \circ A) \cdot ((A \circ A) \cdot 1_n)))$$

$$+2160 \quad 1'_n \cdot ((A \circ A \circ A \circ A \circ A \circ A \circ ((A \circ A \circ (A \cdot A)) \cdot A)) \cdot 1_n)$$

$$+720 \quad 1'_n \cdot ((A \circ A \circ A \circ A \circ A \circ A \circ ((A \cdot A) \cdot (A \circ A \circ A))) \cdot 1_n)$$

$$+40 \quad 1'_n \cdot ((A \circ A \circ A \circ A \circ A \circ A \circ ((A \cdot (A \circ A \circ A)) \cdot A)) \cdot 1_n)$$

$$+720 \quad 1'_n \cdot (((((A \circ A) \cdot 1_n) \circ ((A \circ A \circ A \circ A) \cdot 1_n)) \circ ((A \circ A \circ A \circ A) \cdot 1_n))$$

$$+360 \quad 1'_n \cdot (((A \circ A \circ A \circ A) \cdot ((A \circ A) \cdot 1_n)) \circ ((A \circ A \circ A \circ A) \cdot 1_n))$$

$$+3600 \quad 1'_n \cdot ((A \circ A \circ A \circ A \circ A \circ (A \cdot A) \circ ((A \circ A \circ A) \cdot A)) \cdot 1_n)$$

$$+2520 \quad 1'_n \cdot (((((A \circ A) \cdot 1_n) \circ ((A \circ A \circ A \circ A) \cdot 1_n)) \circ ((A \circ A) \cdot (A \circ A))) \cdot 1_n))$$

$$+360 \quad 1'_n \cdot (((A \circ A \circ A \circ A) \cdot 1_n) \circ ((A \circ A \circ ((A \circ A) \cdot (A \circ A))) \cdot 1_n))$$

$$+560 \quad 1'_n \cdot ((A \circ A \circ A \circ A \circ A \circ ((A \circ A) \cdot d((A \circ A) \cdot 1_n) \cdot (A \circ A))) \cdot 1_n)$$

$$+180 \quad 1'_n \cdot ((A \circ A \circ A \circ A \circ A \circ ((A \circ A) \cdot ((A \circ A) \cdot (A \circ A)))) \cdot 1_n)$$

$$+20 \quad 1'_n \cdot (((A \circ A) \cdot ((A \circ A \circ A \circ A) \cdot 1_n)) \circ ((A \circ A \circ A \circ A) \cdot 1_n))$$

$$+380 \quad \sum_{i_1 i_2 i_3 i_4} (A \circ A \circ A \circ A)_{i_1 i_2} A_{i_1 i_3} A_{i_1 i_4} A_{i_2 i_3} A_{i_2 i_4} (A \circ A)_{i_3 i_4}$$

$$+1080 \quad 1'_n \cdot ((A \circ A \circ A \circ A \circ ((A \circ A \circ ((A \circ A \circ A) \cdot A)) \cdot A)) \cdot 1_n)$$

$$+1760 \quad 1'_n \cdot ((A \circ A \circ A \circ A \circ ((A \circ A \circ (A \cdot A)) \cdot (A \circ A \circ A))) \cdot 1_n)$$

$$+1600 \quad \sum_{i_1 i_2 i_3 i_4} (A \circ A \circ A)_{i_1 i_2} (A \circ A)_{i_1 i_3} A_{i_1 i_4} A_{i_2 i_3} (A \circ A)_{i_2 i_4} A_{i_3 i_4}$$

$$\begin{aligned}
-60 & \quad 1'_n \cdot ((A \circ A \circ ((A \cdot A) \cdot d((A \circ A) \cdot 1_n) \cdot A) \circ (A \cdot (A \cdot A))) \cdot 1_n) \\
-20 & \quad 1'_n \cdot ((A \circ A \circ (((A \cdot A) \cdot A) \circ (A \cdot (A \cdot A))) \cdot (A \circ A))) \cdot 1_n) \\
-20 & \quad 1'_n \cdot ((A \circ A \circ (((A \cdot A) \circ (A \cdot A)) \cdot ((A \cdot A) \circ (A \cdot A)))) \cdot 1_n) \\
-10 & \quad 1'_n \cdot (((A \circ A) \cdot 1_n) \circ ((A \circ ((A \cdot A) \cdot d((A \circ A) \cdot 1_n) \cdot A)) \cdot ((A \circ A) \cdot 1_n))) \\
-20 & \quad 1'_n \cdot (((A \circ A) \cdot 1_n) \circ ((A \circ ((A \cdot A) \cdot A)) \cdot ((A \circ A) \cdot ((A \circ A) \cdot 1_n)))) \\
-20 & \quad 1'_n \cdot (((A \circ A) \cdot ((A \circ (A \cdot A)) \cdot ((A \circ A) \cdot 1_n))) \circ ((A \circ (A \cdot A)) \cdot 1_n)) \\
-60 & \quad 1'_n \cdot (((A \circ A) \cdot 1_n) \circ ((A \circ ((A \cdot A) \cdot (A \circ (A \cdot A) \circ (A \cdot A)))) \cdot 1_n)) \\
-20 & \quad 1'_n \cdot (((A \circ (A \cdot A)) \cdot ((A \circ A) \cdot 1_n)) \circ ((A \circ (A \cdot A)) \cdot ((A \circ A) \cdot 1_n))) \\
-10 & \quad 1'_n \cdot (((A \circ A) \cdot 1_n) \circ ((A \circ ((A \cdot A) \cdot d((A \circ A) \cdot ((A \circ A) \cdot 1_n)) \cdot A)) \cdot 1_n)) \\
-5 & \quad 1'_n \cdot (((A \circ A) \cdot ((A \circ A) \cdot ((A \circ (A \cdot A)) \cdot 1_n))) \circ ((A \circ (A \cdot A)) \cdot 1_n)) \\
-80 & \quad 1'_n \cdot (((A \circ (A \cdot A)) \cdot 1_n) \circ ((A \circ (A \cdot A) \circ (A \cdot A) \circ (A \cdot A)) \cdot 1_n)) \\
-30 & \quad 1'_n \cdot ((A \circ (A \cdot d((A \circ (A \cdot A)) \cdot 1_n) \cdot A) \circ (A \cdot A) \circ (A \cdot A)) \cdot 1_n) \\
-160 & \quad 1'_n \cdot ((A \circ (A \cdot A) \circ ((A \circ ((A \cdot A) \cdot A)) \cdot (A \circ (A \cdot A)))) \cdot 1_n) \\
-80 & \quad 1'_n \cdot ((A \circ ((A \circ (A \cdot A)) \cdot ((A \cdot A) \circ (A \cdot A))) \circ (A \cdot A)) \cdot 1_n)
\end{aligned}$$

Unique graphs for k=8: (27 graphs)

$$\begin{aligned}
+10 & \quad 1'_n \cdot ((A \circ A \circ A \circ (((((A \cdot A) \cdot A) \cdot A) \cdot A) \cdot A) \cdot A)) \cdot 1_n) \\
+20 & \quad 1'_n \cdot (((A \circ A) \cdot 1_n) \circ ((A \circ A) \cdot 1_n)) \circ ((A \circ (((A \cdot A) \cdot A) \cdot A) \cdot A)) \cdot 1_n) \\
+10 & \quad 1'_n \cdot (((A \circ A) \cdot ((A \circ A) \cdot 1_n)) \circ ((A \circ (((A \cdot A) \cdot A) \cdot A) \cdot A)) \cdot 1_n) \\
+40 & \quad 1'_n \cdot (((A \circ A) \cdot 1_n) \circ ((A \circ (((A \cdot A) \cdot A) \cdot A)) \cdot 1_n)) \circ ((A \circ (A \cdot A)) \cdot 1_n) \\
+20 & \quad 1'_n \cdot (((A \circ A) \cdot 1_n) \circ ((A \circ ((A \cdot A) \cdot A)) \cdot 1_n)) \circ ((A \circ ((A \cdot A) \cdot A)) \cdot 1_n) \\
+30 & \quad 1'_n \cdot ((A \circ A \circ (((((A \cdot A) \cdot A) \cdot A) \cdot A) \cdot A) \circ (A \cdot A)) \cdot 1_n) \\
+30 & \quad 1'_n \cdot ((A \circ A \circ (((((A \cdot A) \cdot A) \cdot A) \cdot A) \circ (A \cdot (A \cdot A))) \cdot 1_n) \\
+15 & \quad 1'_n \cdot ((A \circ A \circ (((A \cdot A) \cdot A) \cdot A) \circ (A \cdot ((A \cdot A) \cdot A))) \cdot 1_n) \\
+10 & \quad 1'_n \cdot (((A \circ A) \cdot 1_n) \circ ((A \circ (((A \cdot A) \cdot A) \cdot A) \cdot A)) \cdot ((A \circ A) \cdot 1_n)) \\
+20 & \quad 1'_n \cdot (((A \circ (((A \cdot A) \cdot A) \cdot A)) \cdot ((A \circ A) \cdot 1_n)) \circ ((A \circ (A \cdot A)) \cdot 1_n)) \\
+20 & \quad 1'_n \cdot (((A \circ ((A \cdot A) \cdot A)) \cdot ((A \circ A) \cdot 1_n)) \circ ((A \circ ((A \cdot A) \cdot A)) \cdot 1_n)) \\
+20 & \quad 1'_n \cdot (((A \circ (A \cdot A)) \cdot ((A \circ A) \cdot 1_n)) \circ ((A \circ (((A \cdot A) \cdot A) \cdot A)) \cdot 1_n)) \\
+10 & \quad 1'_n \cdot (((A \circ A) \cdot 1_n) \circ ((A \circ (((A \cdot A) \cdot A) \cdot A) \cdot d((A \circ A) \cdot 1_n) \cdot A)) \cdot 1_n) \\
+20 & \quad 1'_n \cdot (((A \circ A) \cdot 1_n) \circ ((A \circ (((A \cdot A) \cdot A) \cdot d((A \circ (A \cdot A)) \cdot 1_n) \cdot A)) \cdot 1_n)) \\
+10 & \quad 1'_n \cdot (((A \circ A) \cdot 1_n) \circ ((A \circ ((A \cdot A) \cdot d((A \circ ((A \cdot A) \cdot A)) \cdot 1_n) \cdot A)) \cdot 1_n)) \\
+5 & \quad 1'_n \cdot (((A \circ A) \cdot 1_n) \circ ((A \circ (((A \cdot A) \cdot A) \cdot d((A \circ A) \cdot 1_n) \cdot A) \cdot A)) \cdot 1_n) \\
+10 & \quad 1'_n \cdot (((A \circ A) \cdot ((A \circ (((A \cdot A) \cdot A) \cdot A)) \cdot 1_n)) \circ ((A \circ (A \cdot A)) \cdot 1_n)) \\
+20 & \quad 1'_n \cdot (((A \circ ((A \cdot A) \cdot A)) \cdot 1_n) \circ ((A \circ (A \cdot A)) \cdot 1_n)) \circ ((A \circ (A \cdot A)) \cdot 1_n) \\
+30 & \quad 1'_n \cdot ((A \circ (((A \cdot A) \cdot A) \cdot A) \cdot A) \circ (A \cdot A) \circ (A \cdot A)) \cdot 1_n) \\
+60 & \quad 1'_n \cdot ((A \circ (((A \cdot A) \cdot A) \cdot A) \circ (A \cdot (A \cdot A)) \circ (A \cdot A)) \cdot 1_n) \\
+10 & \quad 1'_n \cdot (((A \circ ((A \cdot A) \cdot A)) \cdot ((A \circ (A \cdot A)) \cdot 1_n)) \circ ((A \circ (A \cdot A)) \cdot 1_n)) \\
+20 & \quad 1'_n \cdot (((A \circ (A \cdot A)) \cdot ((A \circ ((A \cdot A) \cdot A)) \cdot 1_n)) \circ ((A \circ (A \cdot A)) \cdot 1_n)) \\
+5 & \quad 1'_n \cdot (((A \circ ((A \cdot A) \cdot d((A \circ (A \cdot A)) \cdot 1_n) \cdot A)) \cdot 1_n) \circ ((A \circ (A \cdot A)) \cdot 1_n)) \\
+5 & \quad 1'_n \cdot (((A \circ A) \cdot ((A \circ ((A \cdot A) \cdot A)) \cdot 1_n)) \circ ((A \circ ((A \cdot A) \cdot A)) \cdot 1_n)) \\
+10 & \quad 1'_n \cdot ((A \circ ((A \cdot A) \cdot A) \circ (A \cdot (A \cdot A)) \circ ((A \cdot A) \cdot A)) \cdot 1_n) \\
+10 & \quad 1'_n \cdot ((A \circ (((A \cdot A) \cdot A) \cdot A) \circ (A \cdot A) \circ (A \cdot A)) \cdot A)) \cdot 1_n) \\
+15 & \quad 1'_n \cdot ((A \circ (((A \cdot A) \cdot A) \cdot A) \circ (A \cdot (A \cdot A)) \circ (A \cdot A)) \cdot A)) \cdot 1_n)
\end{aligned}$$

Unique graphs for k=9: (4 graphs)

$$\begin{aligned}
-10 & \quad 1'_n \cdot (((A \circ A) \cdot 1_n) \circ ((A \circ (((((A \cdot A) \cdot A) \cdot A) \cdot A) \cdot A) \cdot A)) \cdot 1_n)) \\
-10 & \quad 1'_n \cdot (((A \circ (((((A \cdot A) \cdot A) \cdot A) \cdot A) \cdot A) \cdot A)) \cdot 1_n) \circ ((A \circ (A \cdot A)) \cdot 1_n)) \\
-10 & \quad 1'_n \cdot (((A \circ (((A \cdot A) \cdot A) \cdot A) \cdot A)) \cdot 1_n) \circ ((A \circ ((A \cdot A) \cdot A)) \cdot 1_n)) \\
-5 & \quad 1'_n \cdot (((A \circ (((A \cdot A) \cdot A) \cdot A)) \cdot 1_n) \circ ((A \circ (((A \cdot A) \cdot A) \cdot A)) \cdot 1_n))
\end{aligned}$$

Unique graphs for k=10: (1 graphs)

$$+1 \quad 1'_n \cdot ((A \circ (((((((A \cdot (A \cdot A)) \cdot A) \cdot A) \cdot A) \cdot A) \cdot A) \cdot A)) \cdot 1_n)$$

D.9 $m = 11$ and $m = 12$

These formulas are too long to present in the paper. They can be found in the Supplementary Material.

Stage 2 User Prompt: Pruning Process

YOUR TASK (Step 2 of 2): Using the Step 1 output, implement the PRUNING PROCESS that converts each FS term into a computationally efficient SEA or IFS expression.

STEP 2: PRUNING PROCESS (FS terms $\rightarrow$ SEA/IFS terms $\rightarrow$ Final Formula)

An FAM is represented by:

- A dictionary `node_label[v]`, where each node `v` has a vector label. Initially every node label is `ones_n`.
- A list of oriented edge records (`src`, `dst`, `matrix_label`). The edge contributes `matrix_label[i_src, i_dst]` to the Full Sum.
- Parallel edges are allowed. Do not assume that matrix labels are symmetric after pruning.

Initialization from a Stage-1 `weight_matrix`:

- For each pair `u < v` with `weight_matrix[u][v] = w > 0`, create one oriented edge record (`u`, `v`, `A.hadamard_w`), where `A.hadamard_w` means `A` Hadamard-powered `w` times.
- This initial orientation `u  $\rightarrow$  v` is arbitrary but fixed. It is valid because the initial labels are symmetric.
- After pruning, newly created matrix labels may be non-symmetric, so every later access must respect the stored orientation.

Orientation helper:

For an edge between nodes `a` and `b` with stored record (`src`, `dst`, `M`):

- If `src == a` and `dst == b`, then the matrix oriented from `a` to `b` is `M`.
- If `src == b` and `dst == a`, then the matrix oriented from `a` to `b` is `M.T`.

This rule must be used whenever a pruning formula requires a matrix oriented in a specific direction.

2(a) Identify pendants.

- A Type-I pendant has exactly 1 distinct neighbor, ignoring edge directions.
- A Type-II pendant has exactly 2 distinct neighbors, ignoring edge directions.

2(b-I) Type-I pruning.

Suppose `p` is the pendant and `h` is the hinge. Let `v.p` be the node label of `p` and `u.h` be the node label of `h`.

Collect all edge labels between `h` and `p`, oriented from `h` to `p` using the orientation helper.

Let `M.prod` be their Hadamard product.

Update the hinge label by

$$\text{node_label}[h] = \text{node_label}[h] * (M.\text{prod} @ \text{node_label}[p])$$

where `*` is element-wise vector product and `@` is matrix-vector multiplication.

Remove `p` and all edges incident to `p`.

2(b-II) Type-II pruning.

Suppose `p` is the pendant and `x,y` are the two hinges. Use a deterministic order for the hinges, for example `x < y`.

Collect all edge labels between `p` and `x`, oriented from `p` to `x`. Let their Hadamard product be `Q`.

Collect all edge labels between `p` and `y`, oriented from `p` to `y`. Let their Hadamard product be `R`.

Create a new oriented edge from `x` to `y` with label

$$E = Q.T @ \text{diag}(\text{node_label}[p]) @ R$$

Remove `p` and all edges incident to `p`. Add the new edge record (`x`, `y`, `E`).

Important:

- Do not assume `E` is symmetric.
- Do not replace an existing `x-y` edge by `E` unless you correctly preserve the product of all parallel edge labels.
- The safest implementation is to keep `E` as an additional parallel oriented edge record.

Repeat until:

- If only 1 node remains, output/evaluate an SEA term.
- If no pendant remains and more than one node remains, output/evaluate an IFS term.

IFS rule:

For an IFS term, each oriented edge record (`src`, `dst`, `M`) contributes `M[index[src], index[dst]]`. Do not symmetrize `M`. Do not replace `M[index[src], index[dst]]` by `M[index[dst], index[src]]` unless the stored orientation requires a transpose through the orientation helper.

>>> CODE OUTPUT <<<

Produce a self-contained Python script defining `C_m(A, m)` that works for any `m >= 2`.

REMINDERS:

- Edge orientation matters after pruning.
- Initial labels are symmetric, but pruning can create non-symmetric matrix labels.
- Type-I pruning requires all incident edge labels oriented from hinge to pendant.
- Type-II pruning requires `Q` and `R` oriented from pendant to each hinge before computing `Q.T @ diag(v.p) @ R`.
- Keep parallel edges unless you are absolutely sure the Hadamard combination preserves orientation.

-- Carefully distinguish `@`, `*`, Hadamard product, transpose, and `diag`.
 -- For IFS terms, use directed edge indices exactly as stored.

Figure 12: Stage 2 (Pruning Process)⁴² of the benchmark evaluation prompt.